



A Scalable Vulnerability Detection System with Multi-View Graph Representations

SHIHAN DOU, Fudan University, China

HUIYUAN ZHENG, Fudan University, China

JUNJIE SHAN, The University of Hong Kong, China

YUEMING WU, Huazhong University of Science and Technology, China

DEQING ZOU, Huazhong University of Science and Technology, China

XUANJING HUANG, Fudan University, China

YANG LIU, Nanyang Technological University, Singapore

Deep learning (DL) has been extensively utilized in source code vulnerability detection due to its robust automatic feature extraction capabilities. To achieve scalable vulnerability scanning, some prior studies intend to process the source code directly by treating them as text. However, these text-based methods can not achieve the best performance due to the lack of utilizing program semantics. In contrast, other approaches aim to enhance accuracy by distilling program semantics into graph representations for vulnerability detection. Despite their accuracy, these graph-based methods are not scalable, as graph analysis is generally time-intensive.

In this paper, we aim to achieve both scalability and accuracy in scanning large-scale source code vulnerabilities. Inspired by existing DL-based image classification which has the ability to analyze millions of images accurately, we prefer to use these techniques to accomplish our purpose. Specifically, we propose an innovative approach that efficiently converts the source code of a function into multi-view images, while preserving the diverse program information from different perspectives. Based on this approach, we introduce a vulnerability detection system, *VulCNN*, which can leverage the function images and simple convolutional neural networks (CNN) to effectively and efficiently detect vulnerabilities. We implement *VulCNN* and evaluate it on three popular and widely used datasets (*i.e.*, the *Devign*, *Big-Vul*, and *ReVeal* datasets). Experimental results report that *VulCNN* can achieve better accuracy than nine state-of-the-art vulnerability detectors (*i.e.*, *TokenCNN*, *VulDeePecker*, *SySeVR*, *VulDeeLocator*, *Devign*, *EPVD*, *VulDecgre*, *UniXcoder*, and *PDBERT*). We also compare *VulCNN* with three widely used Large Language Models (*i.e.*, *Llama-2-Instruct*, *DeepSeek-Coder*, *Qwen2.5-Coder-Instruct*) and one LLM-based detection method (*i.e.*, *GRACE*), and the experimental results demonstrate that our proposed method can perform better than most of vulnerability detection methods. As for scalability, *VulCNN* is about five times faster than *VulDeeLocator*, and about two times faster than *Devign*. *VulCNN* achieves vulnerability detection effectiveness comparable to the state-of-the-art method *GRACE*, but with a 31.52% reduction in time overhead.

CCS Concepts: • **Security and privacy** → **Vulnerability scanners**.

Additional Key Words and Phrases: Vulnerability Detection, CNN, Large Scale, Image

Authors' addresses: Yueming Wu is the corresponding author, and the first two authors contributed equally. S. Dou, H. Zheng, and X. Huang are with Fudan University, China; emails: shihandou@foxmail.com, hyzheng24@m.fudan.edu.cn, xjhuang@fudan.edu.cn. J. Shan is with The University of Hong Kong, China; email: junjieshan@connect.hku.hk. Y. Wu and D. Zou are with Huazhong University of Science and Technology, China; emails: yuemingwu@hust.edu.cn, deqingzou@hust.edu.cn. Y. Liu is with Nanyang Technological University, Singapore; email: yangliu@ntu.edu.sg.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s).

ACM 1557-7392/2025/10-ART

<https://doi.org/10.1145/3770075>

1 INTRODUCTION

Recently, various cyberspace security incidents [2, 3] such as hacker extortion, botnet attacks, and user information leakage have occurred frequently. As an important part of cyberspace, vulnerabilities in software systems have brought serious security threats to cyberspace. In 2020, Synopsys *Open Source Security and Risk Analysis* (OSSRA) examined audit data from 1,250+ commercial codebases and revealed that 70% of the code in these codebases were open-source. Moreover, 75% of these open-source codebases contained open-source security vulnerabilities, and nearly half contained high-risk vulnerabilities [1]. Therefore, it is urgent to carry out large-scale and intelligent software vulnerability detection methods to better protect software security.

In general, source code vulnerability detection methods can be divided into two main categories, that is, the code-similarity-based methods [42, 48, 53, 56, 71] and the pattern-based methods [5, 6, 9, 30, 58, 59, 77, 102, 103]. Vulnerability detection methods based on code similarity are mainly used to detect vulnerabilities caused by code cloning. When used to detect vulnerabilities not caused by code cloning, it will lead to a high false negative rate [58]. Traditional pattern-based vulnerability detection methods [5, 6, 9] rely on experts to manually define vulnerability rules or characteristics to describe vulnerabilities. These approaches are not only subjective but also difficult to achieve a low false positive rate and a low false negative rate at the same time [57, 58].

In recent years, due to the automatic feature extraction of *deep learning* (DL), it has been widely used to detect source code vulnerability. These DL-based techniques [30, 38, 57–59, 77, 85, 98, 102, 103] belong to the second category of methods (*i.e.*, pattern-based methods). They do not require experts to manually define features and can automatically generate vulnerability patterns. For example, some prior studies [58, 77] treat the program source code as text and apply techniques in the field of natural language processing to detect vulnerability. The detection performance of these text-based methods is not ideal since they ignore the program semantics of the source code. To address the issue, researchers conduct program analysis to distill the program semantics of source code into a graph representation and perform graph analysis (*e.g.*, graph neural network) to detect vulnerability. These graph-based techniques [30, 98, 102] can achieve higher effectiveness on detecting vulnerability, however, these methods typically employ graph-based analytical techniques such as GNNs that require substantial computational resources, particularly when handling long-range message passing, thereby limiting their scalability [92].

In this paper, we aim to achieve accuracy and scalability simultaneously on detecting vulnerabilities from large-scale source code. Our key idea is derived from DL-based image classification, which can process millions of images while maintaining high accuracy, and the classification results can be interpreted by visualization techniques. Specifically, we mainly address one major challenge in our paper.

- *How to effectively convert the source code of a function into multi-view images while preserving the diverse details of the program from different perspectives?*

To tackle this challenge, we begin by conducting a diverse program analysis to distill diverse program information of a function into multiple graphs. Specifically, we convert the source code into an *abstract syntax tree* (AST), which represents the program's syntactic structure; a *control flow graph* (CFG), which details the program's execution flow; and a *program dependency graph* (PDG), which contains both the control-flow and data-flow dependencies within the source code. After obtaining the AST, CFG, and PDG of a function, we treat a single graph as a social network and apply centrality analysis on the network to attach the graph's structural information to the source code. In social network analysis, centrality analysis [34, 47] has been proposed to measure the importance of a node within the network. Specifically, we leverage six different centralities (*i.e.*, degree centrality [34], katz centrality [47], betweenness centrality [18], eigenVector centrality [17], harmonic centrality [75], and closeness centrality [34]) to commence our image transformation. There are two main reasons for using multiple centralities. First, different centralities can maintain the graph properties from different aspects [90]. Second, an image generally has multiple channels (*i.e.*, red, green, and blue) and they work together to produce a complete image. The output of centrality analysis is an image while preserving the graph details

from multiple diverse aspects. Given generated function images, we then train a *convolutional neural network* (CNN) [50, 51] model and use it to detect vulnerability.

We implement our proposed vulnerability detection system *VulCNN* and evaluate it on three widely used datasets (*i.e.*, the *Devign* [102], *Big-Vul* [32], and *ReVeal* datasets [20]). We compare our method with nine popular vulnerability detectors (*i.e.*, *TokenCNN* [77], *VulDeePecker* [58], *SySeVR* [57], *VulDeeLocator* [55], *Devign* [102], *EPVD* [98], *VulDecgre* [43], *UniXcoder* [36], and *PDBERT* [62]). Evaluation results show that *VulCNN* can achieve better effectiveness than these widely used vulnerability detectors. We also compare it against three large language models (LLM) (*i.e.*, *Llama-2-Instruct* [85], *DeepSeek-Coder* [38], and *Qwen2.5-Coder-Instruct* [41]) and one SOTA LLM-based detection system (*i.e.*, *GRACE* [64]). Experimental results demonstrate that *VulCNN* can detect more vulnerabilities than these LLMs, and also achieves comparable performance than *GRACE*. Furthermore, we also evaluate the detection efficiency of these systems. We find that *VulCNN* is about two times faster than the state-of-the-art graph-based vulnerability detection tool *Devign*. *VulCNN* is approximately 31.52% faster and does not incur any token-related costs at inference time compared to the state-of-the-art vulnerability detection system *GRACE*. These results indicate that *VulCNN* achieves more effective and efficient performance in vulnerability detection than other popular detectors.

In summary, this paper makes the following contributions:

- We propose a novel idea that can efficiently convert the source code of a function into multi-view images while preserving the program details from different perspectives.
- We design and implement a prototype system, *VulCNN*¹, a scalable graph-based vulnerability detection system.
- We conduct evaluations on three popular datasets (*i.e.*, *Devign*, *Big-Vul*, and *ReVeal* dataset). Experimental results report that *VulCNN* is superior to nine state-of-the-art vulnerability detectors (*i.e.*, *TokenCNN*, *VulDeePecker*, *SySeVR*, *VulDeeLocator*, *Devign*, *EPVD*, *VulDecgre*, *UniXcoder*, and *PDBERT*), three LLM-based detection systems (*i.e.*, *Llama-2-Instruct*, *DeepSeek-Coder*, and *Qwen2.5-Coder-Instruct*), and one SOTA LLM-based vulnerability detector (*i.e.*, *GRACE*).
- We conduct extensive experiments to evaluate the contributions of the various components of *VulCNN*. We also investigate which graph structures and which centrality analysis approaches are most effective and valuable for our vulnerability detection system.

This paper is an extended version of our previous work [91] published in IEEE/ACM 44th International Conference on Software Engineering (ICSE) 2022. We significantly enhanced the previous conference version in the following aspects: I) We updated the graph extraction framework by converting the source code of a function into multi-view images (Section 3.2). This approach preserves the diverse details of the program from different perspectives. II) We added additional three centrality analysis approaches (*i.e.*, harmonic centrality, betweenness centrality, and eigenVector centrality) to further analyze the function graph representations (Section 3.3). This allows us to obtain more detailed information regarding the importance of nodes within the function graph. III) We enhanced the classification algorithm by introducing a learnable adaptive re-weight layer (Section 3.4). This modification provides the model with greater flexibility, enabling it to autonomously learn the optimal weighted combination of the three representations, rather than merely averaging or directly summing them. As a result, the classifier can adaptively adjust the influence of each image, thereby better accommodating different types of vulnerabilities. IV) The training and test sets used in the previous conference version were constructed by ourselves. To further evaluate our method, we conducted comparative experiments using three recent and widely utilized datasets: *Devign*, *Big-Vul*, and *ReVeal* datasets. We demonstrated the effectiveness of *VulCNN* on extensive experiments (Section 4.2). V) We added eight additional SOTA methods for comparison, including two graph-based detection systems, two pre-trained code representation models, three LLMs, and one LLM-based

¹<https://github.com/Ablustrund/VulCNNPlus>

detection system. (Section 4.2). VI) We conducted extensive ablation experiments to explore and illustrate the impact of the multi-view images and centrality analysis methods for vulnerability detection (Section 4.3). VII) We analyzed the time consumption of the different modules of *VulCNN*. Additionally, we compared the time required by our method and other SOTA methods for detecting vulnerabilities (Section 4.4).

Paper organization. The remainder of the paper is organized as follows. Section 2 presents the motivation of our paper. Section 3 introduces our system. Section 4 reports the experimental results. Section 5 discusses the future work. Section 6 describes the related work. Section 7 concludes the present paper.

2 MOTIVATION

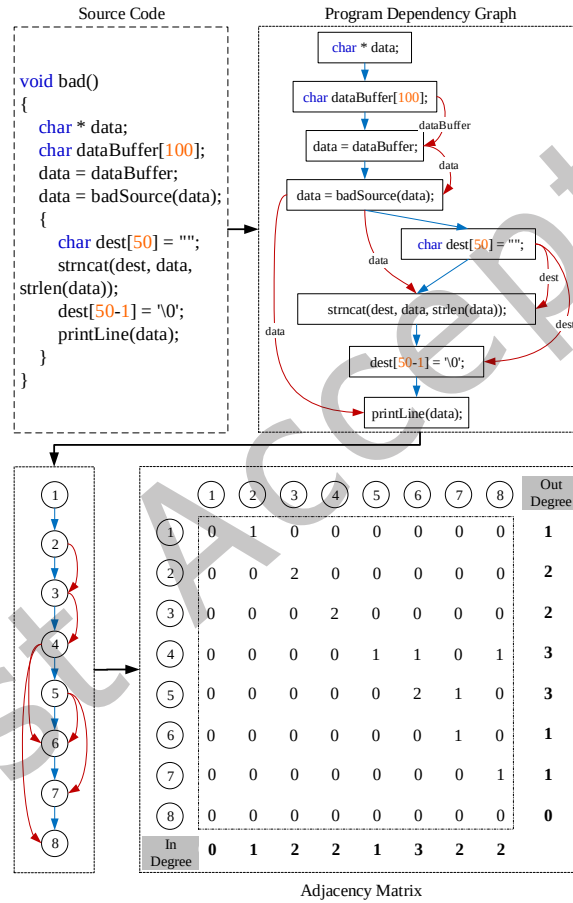


Fig. 1. The *program dependency graph* (PDG) and corresponding adjacency matrix of a vulnerability

In this section, we illustrate our method motivation for converting the source code of a function into multi-view images. We all know that we can carve out the syntactic structure information and program semantics (*i.e.*, functionality) of a function by using program analysis. A function graph obtained through program analysis, such as an AST or PDG, contains multiple nodes, which together implement the function's syntax or semantic

information. However, the contribution or importance of different nodes is different. For example, some nodes just involve the definition of variables, while some nodes involve the core execution process of the function. Obviously, the latter is more important. Therefore, we raise a question: “How to model the different contributions of different nodes to the function graph?”

To answer it, we select a buffer overflow vulnerability function as our example and use the *program dependency graph* (PDG) to illustrate our motivation. A PDG contains both control-flow and data-flow details, which can carve out the program semantics of a function. Specifically, we leverage *Joern* [93] to obtain the PDG of the vulnerability in Figure 1. Each node in the PDG corresponds to a line of code in the vulnerability. Red lines and blue lines show the data flows and control flows between different lines of code in the function, respectively. To simply display the PDG of the vulnerability, we replace each line of code with a numbered circular node. Eight lines of code in the function correspond to eight circular nodes as shown in Figure 1. In graph theory, an adjacency matrix is a square matrix used to represent a finite graph [4]. Therefore, to represent the PDG of the vulnerability, we compute the corresponding adjacency matrix and describe it in Figure 1. The elements of the matrix indicate whether pairs of nodes are adjacent or not in the graph. Since our PDG is a directed graph, an element represents the number of directed edges between two different nodes. For example, node 3 has two edges pointing to node 4, while node 4 has no edges pointing to node 3. Therefore, the corresponding values between node 3 and node 4 in the matrix are two and zero, respectively.

After obtaining the adjacency matrix of the PDG, we find that when we sum a row in the matrix, the value obtained corresponds to the out-degree of the node. Moreover, when we sum a column, the value obtained is the in-degree of the node. For example, the sum of the row of node 4 is three (*i.e.*, $1+1+1=3$), which indicates that the out-degree of node 4 is three. To better illustrate our insight, we present the in-degrees, out-degrees, and degrees of all nodes (*i.e.*, all lines of code of the vulnerability) in Figure 2. Through the results in Figure 2, we find that the degrees of different nodes are basically different.

It is reasonable because there are different relationships (*i.e.*, control-flows and data-flows) between different nodes, and the vulnerability is triggered based on these relationships. If we directly process these codes by treating them as text, the degrees of all lines of code are one, which may decrease the vulnerability detection accuracy.

In one word, a graph can be represented by its adjacency matrix, the matrix can be described by the degrees of all nodes. Therefore, computing the degrees of nodes in a graph may be a great candidate to retain the graph details. In practice, the degree of a node in a graph is originally used to quantify its importance. It has been widely used in social network analysis [34]. The higher the degree, the more important the person. Meanwhile, different nodes in a graph have different degrees. If we treat each node as a person, the control-flow and data-flow relationships as the communications between persons, then the corresponding PDG can be regarded as a social network. The higher the degree of a person, the more other persons communicated with him, and the greater his importance within the PDG social network. Therefore, we can leverage the importance of a node in a PDG as a form of the contribution of program semantics. In other words, the more important a node is, the more it may contribute to implementing the program semantics (*i.e.*, functionality) of a graph.

Furthermore, we can generate distinct graphs of a function by employing various program analysis techniques. Each type of graph provides a unique perspective on the function’s details. Within these graphs, nodes possess varying semantics and levels of importance. By applying centrality analysis, these semantically diverse function graphs can be converted into different images. These images collectively characterize a function from multiple viewpoints, offering a comprehensive portrayal that enhances the potential to detect vulnerabilities within the function. Based on these observations, we design *VulCNN* by analyzing the importance of nodes in diverse function graphs to detect the vulnerability.

Node		In Degree	Out Degree	Degree
1	{			
2	char * data;	0	1	1
3	char dataBuffer[100];	1	2	3
4	data = dataBuffer;	2	2	4
5	data = badSource(data);	2	3	5
6	{ char dest[50] = "";	1	3	4
7	strncat(dest, data, strlen(data));	3	1	4
8	dest[50-1] = '\0';	2	1	3
9	println(data);	2	0	2
10	}			

Fig. 2. The in-degrees, out-degrees, and degrees of all nodes in Figure 1

3 SYSTEM

In this section, we introduce *VulCNN*, a novel and efficient source code vulnerability detection system.

3.1 Overview

As shown in Figure 3, *VulCNN* consists of four main phases: *Graph Extraction*, *Node Embedding*, *Image Generation*, and *Classification*.

- **Graph Extraction:** Given the source code of a function, we first normalize them and then perform static analysis to extract the abstract syntax tree, the control flow graph, and the program dependency graph of the function.
- **Node Embedding:** Each function graph of a function (*i.e.*, AST, CFG, and PDG) has multiple nodes. We regard specific information of each node as a text and embed them into a vector.
- **Image Generation:** After embedding all nodes of the function graphs, we apply centrality analysis to obtain the importance of all nodes and multiply them by the vectors one by one. For each function graph, we can obtain an image which embeds the program details.
- **Classification:** Our final phase focuses on classification. Given generated multi-view images, we first train a parameter-shared CNN model and then use it to detect vulnerability.

3.2 Graph Extraction and Node Embedding

VulCNN aims to achieve accuracy and scalability simultaneously on detecting vulnerabilities. Therefore, we first apply static analysis approaches to distill the program information of source code into multiple graph representations from different perspectives. Due to the coarse granularity of file-level vulnerability detection, we focus on detecting vulnerability at a more fine-grained level (*i.e.*, function-level) since a function can also implement a specific task. Before extracting the graph representations of a function, we first abstract and normalize the source code. Particularly, we use three levels of normalization which make *VulCNN* resilient to common code modifications while preserving program semantics.

Figure 4 shows the detailed transformations of a function at varying normalization levels.

- **Step1:** Remove the comments in the source code because they have nothing to do with program semantics.
- **Step2:** Map user-defined variables to symbolic names in a one-to-one manner (*e.g.*, VAR1).
- **Step3:** Map user-defined functions to symbolic names in a one-to-one manner (*e.g.*, FUN1).

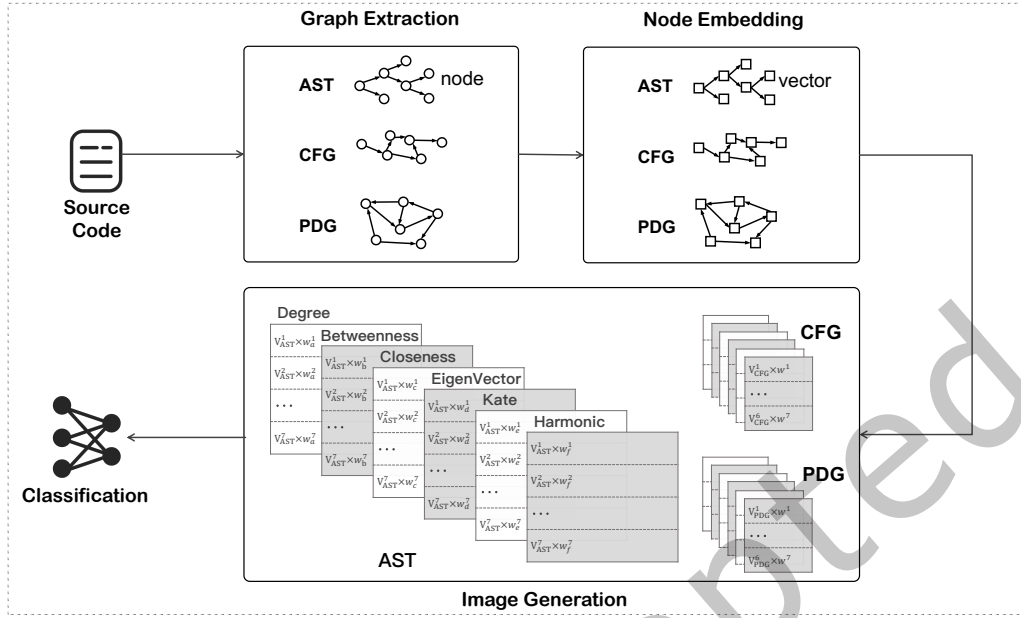
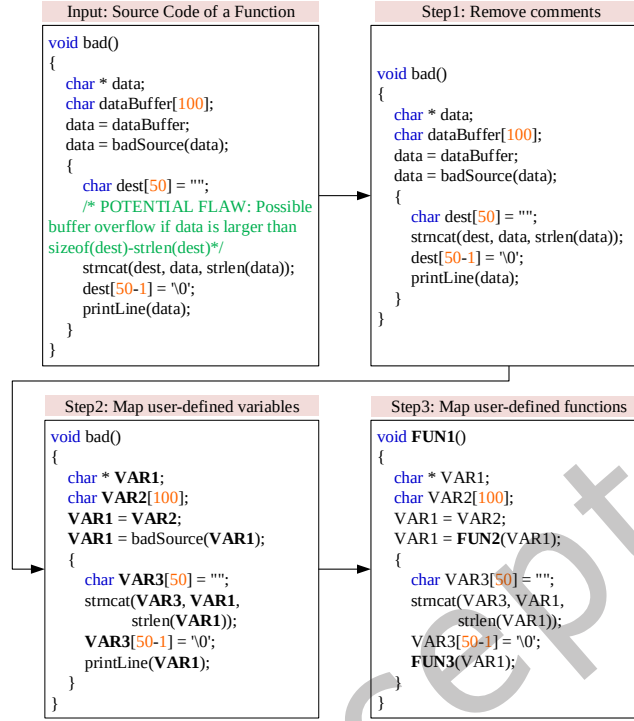


Fig. 3. System overview of VulCNN

After abstracting the source code, we then leverage an open-source code analysis platform for C/C++, *Joern* [8, 93], to extract the *abstract syntax tree* (AST), the *control flow graph* (CFG), and the *program dependency graph* (PDG) of the function. AST is a tree representation of the abstract syntactic structure of source code. Each node in the tree denotes a construct occurring in the source code. CFG is a graphical representation of all the paths that might be traversed through a program during its execution. Each node in the graph represents a basic block and the edges between the nodes represent the control flow paths. PDG is a graph that contains explicit both the data dependencies and control dependencies between operations of the source code. Each node in a PDG corresponds to a line of code in the function. We treat the information of each node as a text (e.g., the information of each node in PDG is a line of code), and apply text embedding approaches to transform it into a fixed-length vector. Specifically, for CFG and PDG, we utilize a widely used method (i.e., sent2vec [70]) to convert each node (i.e., the line of codes) into a sentence embedding. Sent2vec adopts a simple but efficient unsupervised objective to train distributed representations of sentences. For AST, due to the information of each node being a character, we use the word2vec [74] to transfer each node to a token embedding. Using the sent2vec [70] and word2vec [74] model, we can transform the node into its corresponding vector representation whose dimension is 128 in our paper.

To better illustrate the detailed steps involved in our proposed method, we provide an example which converts a PDG of a function into an image, as shown in Figure 5. In this example, we first extract the PDG from the function that achieves the Fibonacci sequence algorithm. We convert each node in the function graph into a vector. Then, we utilize six centrality analysis methods to analyze this function graph. Finally, we utilize the analysis results and node embeddings to generate a function image. For both the function's AST and CFG, we employ the same process to convert each of them into corresponding images.

Fig. 4. Steps of source code normalization in *VulCNN*

3.3 Image Generation

After graph extraction and node embedding, we obtain three types of graph representations for each function, with each node embedded as a vector. We then convert these function graphs into multi-view images. We treat each graph as a social network and apply centrality analysis to measure node importance, ensuring that the semantic contribution of each node is captured during image generation. Moreover, a real color image consists of multiple channels (*i.e.*, red, green, and blue), which together form a complete image. Inspired by this, we employ diverse centrality analysis approaches to characterize these function graphs to generate multi-channel images from different perspectives.

Table 1 summarizes the definitions and mathematical formulations of the centrality metrics we used, including degree, closeness, harmonic, betweenness, eigenvector, and Katz centralities. Degree centrality quantifies a node's immediate connectivity, highlighting locally influential nodes with many direct neighbors [34]. Closeness centrality and its variant, harmonic centrality [66], measure the average shortest-path distance from a node to all others, reflecting global accessibility and reachability. Betweenness centrality evaluates how often a node appears on the shortest paths between other nodes [18], identifying key intermediary nodes that control or mediate information flow across the graph. Eigenvector centrality assigns higher scores to nodes connected to other influential nodes [17], capturing recursive importance and influence propagation through the network. Katz centrality extends this by considering longer paths with decay factors [47], enabling the model to recognize nodes with indirect influence across larger substructures. These centrality metrics are selected because they emphasize different structural perspectives and are widely used in various tasks [15, 39, 87]. By jointly modeling

Table 1. Definitions and formulas of the centrality analysis methods used in *VulCNN*

Centrality Metric	Definition
Degree Centrality [34]	Measures the fraction of nodes a node is directly connected to. It is normalized by the maximum possible degree ($N - 1$), where N is the number of nodes: $x_i = \frac{deg(i)}{N-1}$ where $deg(i)$ is the degree of node i .
Katz Centrality [47]	Computes the centrality based on the centrality of neighbors, incorporating immediate and distant neighbors: $x_i = \alpha \sum_j A_{ij} x_j + \beta$ where A is the adjacency matrix, β controls the initial centrality, and $\alpha < \frac{1}{\lambda_{\max}}$ with $\lambda_{\max}$ the largest eigenvalue of A .
Closeness Centrality [34]	Measures how close a node is to all other nodes, based on the average shortest path distance: $x_i = \frac{N-1}{\sum_{j \neq i} d(i, j)}$ where $d(i, j)$ is the shortest path distance between nodes i and j .
Betweenness Centrality [18]	Quantifies the number of shortest paths passing through a node: $x_i = \sum_{s \neq i \neq t} \frac{\delta_{st}(i)}{\delta_{st}}$ where δ_{st} is the total number of shortest paths from s to t , and $\delta_{st}(i)$ is the number of those paths passing through i .
Eigenvector Centrality [17]	Assigns higher centrality to nodes connected to other highly central nodes: $x_i = c \sum_{j=1}^n a_{ij} x_j$ where $a_{ij} = 1$ if nodes i and j are connected, and c is a proportionality constant.
Harmonic Centrality [66]	Designed for disconnected graphs, measuring the sum of reciprocal distances: $x_i = \sum_{j=1}^n \frac{1}{dis(i, j)}$ where $dis(i, j)$ is the shortest path distance. If i is unreachable from j , $\frac{1}{dis(i, j)}$ is considered 0.

adopted in image processing tasks due to their ability to extract fine-grained features without manual preprocessing. We use CNNs to extract features from the generated multi-view images of the function, and utilize them to detect potential vulnerabilities.

Since CNNs require fixed-size inputs but graphs of different functions vary in the number of nodes, we unify the image height by setting a threshold. Based on the distribution shown in Figure 6, where more than 99% of functions have fewer than 1,000 nodes, we set the node threshold at 1,000. Functions with fewer nodes are

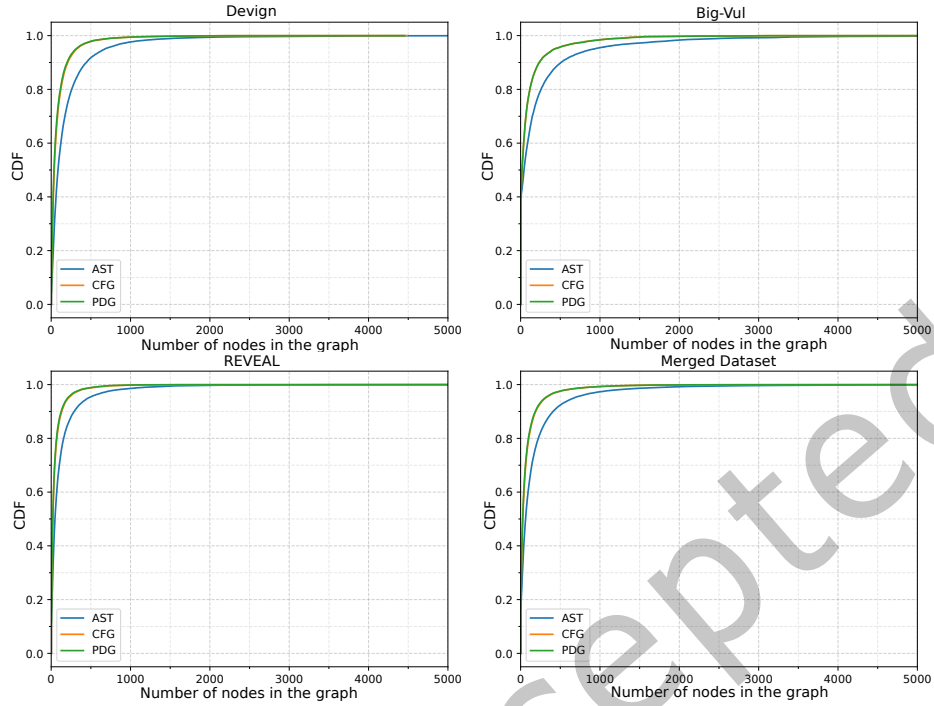


Fig. 6. The Cumulative Distribution Function (CDF) of the total number of nodes in the function's AST, CFG, and PDG, respectively.

zero-padded, and those exceeding the threshold are truncated. Thus, each function's multi-view image has a size of $1000 \times 128 \times 6$ (1,000 nodes, 128-dimensional embeddings, and six channels).

For feature extraction, we employ CNNs with 10 groups of filters of different sizes, each with dimensions of $m \times 128 \times 6$ ($m = 1$ to 10). For each filter size, we utilize 32 filters to capture features at various levels. Max pooling is applied to further extract key features. Finally, each image of the multi-view images is represented as a 320-dimensional vector. We also apply parameter sharing across the CNNs for AST, CFG, and PDG images to conserve memory and maintain consistency.

As each function produces three representation vectors (one per graph view), we introduce an adaptive re-weighting module to combine them, as shown in Figure 7. A fully connected layer learns to assign weights to the three vectors, followed by a softmax normalization. The weighted representations are fused into a single vector, which is fed into a final fully connected layer for classification. This adaptive strategy allows the model to automatically emphasize the most informative views for each function, enabling flexible and effective vulnerability detection across diverse types of program structures.

Overall, this module integrates the multi-view images of a function into a single feature representation and then uses it to determine whether the function contains vulnerabilities.

4 EXPERIMENTS

In this section, we aim to answer the following research questions:

- *RQ1: What is the detection performance of VulCNN when detecting source code vulnerability?*

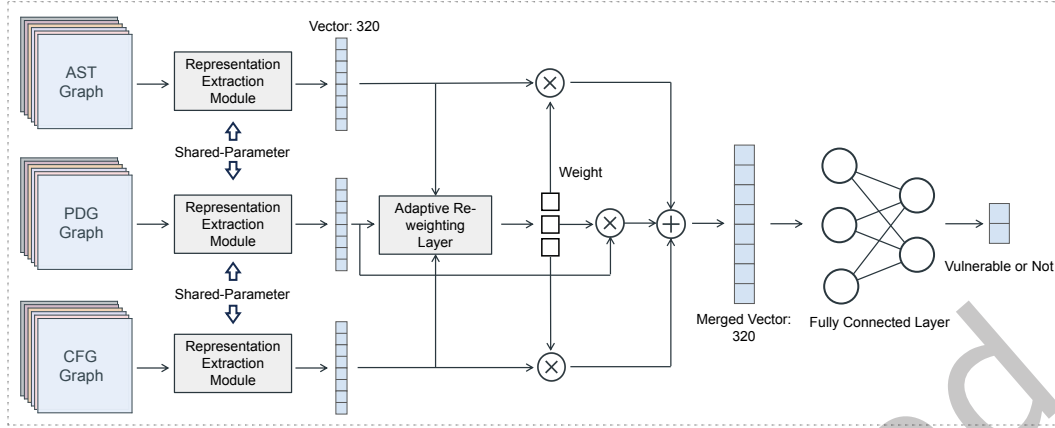


Fig. 7. Classification module of VulCNN

- RQ2: How do different centrality metrics, the adaptive re-weighting layer, and function graph representations contribute to the effectiveness of VulCNN?
- RQ3: What is the runtime overhead of VulCNN when detecting source code vulnerability?
- RQ4: What are the differences in detection performance and computational overhead between pre-trained code embedding models and traditional embedding methods?

4.1 Experiment Setup

4.1.1 Datasets. To evaluate the effectiveness of VulCNN on vulnerability detection, we choose three widely used datasets as shown in Table 2 (i.e., *Devign dataset* [102], *Big-Vul* [32], and *ReVeal dataset* [20]). We construct the training, validation, and test sets from these three dataset. The **Devign dataset** [102] consists of 12,460 vulnerabilities and 14,858 normal functions. A team of four professional security researchers spent 600 man-hours to construct it. Functions in the dataset are mainly obtained from two open-source software, namely qemu [14] and ffmpeg [12]. A total of 7,479 vulnerable functions and 10,070 normal functions are analyzed from qemu, and a total of 4,981 vulnerable functions and 4,788 normal functions are obtained from ffmpeg. The **ReVeal dataset** [20] comprises over 22,000 functions sourced from two extensive open-source projects: the Linux Debian Kernel and Chromium. Of these functions, 9.15% have been identified as vulnerable. The **Big-Vul** dataset [32] is constructed from CVE entries spanning from 2002 to 2019, encompassing 348 distinct projects and 91 unique types of vulnerabilities. The *Big-Vul* dataset is unbalanced, and it contains 177,736 normal samples and 10,900 vulnerabilities.

Data imbalance during training generally has a significant negative impact on DL-based models, not only in vulnerability detection [21, 81] but also in other domains [31, 82]. Specifically, Chakraborty et al. [21] show that training vulnerability detection models on imbalanced datasets leads to learned biases, resulting in model performance that does not align with real-world scenarios; they address this issue through sub-sampling. Shestov et al. [81] also demonstrate that training DL-based models on imbalanced datasets adversely affects vulnerability detection performance, thereby compromising the fairness of comparisons. In more general tasks, Dullerud et al. [31] found that DL-based models trained on imbalanced datasets tend to learn dataset biases, resulting in predictions skewed toward the majority class and hindering effective model evaluation. Moreover, Soleimani et al. [82] show that, without reweighting strategies to address data imbalance, DL-based models are disproportionately influenced by the majority class. These findings suggest that DL-based models should be evaluated on balanced

Table 2. Summary of our used dataset

Dataset	#Vulnerable functions	#NonVulnerable functions	# Ratio of Vulnerable functions
Devign	12,460	14,858	1:1.2 = 0.838
Big-Vul	10,900	177,736	1:16.3 = 0.061
ReVeal	2,240	19,987	1:8.9 = 0.112

datasets to enable more reasonable and meaningful comparisons. Based on these observations, some vulnerability detection studies train models on balanced datasets to further mitigate the adverse effects of data imbalance [19, 43]. For example, Jia et al. [43] demonstrate that imbalanced datasets can negatively impact the performance of vulnerability detection models and thus construct a balanced dataset through sub-sampling. Similarly, Cao et al. [19] employ sampling techniques to balance the training data. Therefore, evaluating models on balanced datasets may better reflect their true effectiveness.

Following prior works [29, 98], we evaluate our proposed method on a merged dataset consisting of three widely adopted benchmarks: *Devign* [102], *Big-Vul* [32], and *ReVeal* [20]. To ensure the integrity of our evaluation and avoid data leakage, we first perform deduplication on all raw samples. Following prior work [98], this involves converting each sample into tokens using a C/C++ lexer and removing its comments, then identifying and removing duplicate samples. After deduplication, we construct a balanced dataset for robust training. Specifically, this dataset merges all samples from *Devign*, all vulnerable samples from *Big-Vul* and *ReVeal*, and a randomly selected subset of non-vulnerable samples from these two datasets. The number of selected non-vulnerable samples is matched to the total number of vulnerable samples to achieve class balance. On this balanced dataset, we split it into ten parts: eight parts are used for training, one for validation, and one for testing. Furthermore, to ensure that the validation and test sets reflect real-world class distributions, we augment them with additional non-vulnerable samples. Specifically, for samples in these sets that come from *Big-Vul* and *ReVeal*, we randomly add non-vulnerable samples so that the proportion of vulnerable samples matches the original ratio in the raw datasets. All samples across the training, validation, and test sets are non-overlapping. This augmentation enables a more realistic evaluation of the model's performance in practical scenarios. Although we utilize the mixed dataset during the training and validation process, we report the performance on each dataset separately to clearly show the effectiveness of *VulCNN*. Moreover, we also evaluate all baseline models and our proposed *VulCNN* on the same training, validation, and test sets to further ensure a fair comparison of results. This experimental setting is consistent with previous vulnerability detection studies [29, 98].

4.1.2 Baselines. Researchers have developed many widely used models for detecting vulnerabilities in functions. A common approach is text-level detection, which relies solely on the source code of a function. Since a program can be represented by various function graphs, such as the abstract syntax tree (AST) and control flow graph (CFG), many studies have also focused on leveraging these graph representations to detect vulnerabilities. Moreover, recently LLM-based detection methods have achieved remarkable performance in vulnerability detection. To comprehensively evaluate the effectiveness of *VulCNN*, we compare it against 13 popular vulnerability detection methods from these different categories. These eight detectors² are as follows:

- **TokenCNN** [77]: A token-based vulnerability detector that processes program tokens obtained through lexical analysis. *TokenCNN* inputs these tokens into a *convolutional neural network* (CNN) to train a model for vulnerability detection.

²For convenience, we name the unnamed models based on their code representations (e.g., token) and neural networks (e.g., CNN).

- **VulDeePecker** [58]: A slice-based vulnerability detector that analyzes program slices extracted through static analysis. *VulDeePecker* inputs these slices into a *bidirectional long short-term memory* (BLSTM) network to train a model for vulnerability detection.
- **SySeVR** [57]: A systematic framework for vulnerability detection in C/C++ programs that leverages deep learning to obtain syntax- and semantics-aware vector representations, demonstrating effectiveness in discovering previously unreported vulnerabilities.
- **VulDeeLocator** [55]: A deep learning-based detector designed to achieve both high vulnerability detection capability and precise location identification by leveraging intermediate code and granularity refinement.
- **Devign** [102]: A graph-based vulnerability detector that extracts AST, CFG, DFG, and *natural code sequence* (NSC) representations of a function. *Devign* utilizes these representations by feeding them into a *graph neural networks* (GNN) to train a model for vulnerability detection.
- **EPVD** [98]: A graph-based vulnerability detector that breaks down code snippets into multiple execution paths. *EPVD* analyzes these paths using a pre-trained code model and a Convolutional Neural Network (CNN), with the resulting representation subsequently utilized by a Multilayer Perceptron (MLP) classifier to detect vulnerabilities, effectively addressing challenges associated with irrelevant information and lengthy code.
- **VulDecgre** [43]: A graph-based vulnerability detector that strengthens graph representations by explicitly modeling global dependency relationships, enabling more accurate vulnerability identification.
- **UniXcoder** [36]: A unified cross-modal pre-training framework designed for code understanding tasks. *UniXcoder* jointly learns from programming language texts and structural code representations, demonstrating strong performance across multiple software engineering tasks.
- **PDBERT** [62]: A pre-training approach that explicitly models program dependency structures. By predicting dependencies during pre-training, the resulting transformer models achieve improved performance on vulnerability detection tasks.
- **Llama-2-Instruct** [85]: An advanced LLM-based detection method developed by Meta-AI, featuring a stack of decoder-only transformers. *Llama-2-Instruct* is trained on extensive text and code corpora, and the 7B parameter version is utilized to conduct all experiments in this study.
- **DeepSeek-Coder** [38]: An LLM-based detection method with a similar architecture to *Llama-2-Instruct*, trained on 2 trillion tokens combining code and natural languages. *DeepSeek-Coder* has been shown effective in code tasks, including detecting vulnerabilities in functions [28, 52, 60, 72]. The 6.7B parameter version is employed in this study.
- **Qwen2.5-Coder-Instruct** [41]: A specialized LLM optimized for software engineering tasks, including vulnerability detection. *Qwen2.5-Coder-Instruct* leverages instruction tuning and domain-specific pretraining to achieve strong performance in vulnerability identification benchmarks.
- **GRACE** [64]: An LLM-based vulnerability detection framework that integrates graph structural information and in-context learning. *GRACE* transforms code property graphs into natural language descriptions and employs retrieval-augmented demonstrations based on semantic, lexical, and syntactic similarity, significantly improving vulnerability detection accuracy.

For slice-based approaches [57, 103], a single function may correspond to multiple slices. To ensure a fair comparison, we perform vulnerability detection on each slice; if any slice of a function is identified as vulnerable, the entire function is classified as vulnerable. This experimental setting is consistent with that used in other vulnerability detection studies [22, 98]. For LLM-based vulnerability detection methods, we adopt two settings to ensure fair comparison. First, for GRACE [64], we follow the experimental setup described in its original paper. Second, for general-purpose LLMs, *i.e.*, Llama-2-Instruct [85], DeepSeek-Coder [38], and Qwen2.5-coder-instruct [41], we follow the experimental setting used in previous related studies [100] to ensure the reliability of results.

The temperature is set to 0.2 and the top-p is set to 0.9 to ensure more stable outputs. On the other hand, Zhou et al. [101] have shown that randomly selecting two samples as demonstrations (*i.e.*, 2-shots) can make LLM outputs more controllable and improve detection effectiveness. Therefore, in each cross-validation fold, we randomly select one benign function and one vulnerable function from the training set as demonstrations. We report the average result of ten tests. Specifically, the prompt used for vulnerability detection is as follows:

Prompt: Now you need to identify whether a method contains a vulnerability or not. If has any potential vulnerability, output: ‘this code is vulnerable’. Otherwise, output: ‘this code is non-vulnerable’. Here are sampled examples from the training data.

Example #1:
 Code:
 {benign example}
 Label:
 this code is non-vulnerable.

Example #2:
 Code:
 {vulnerable example}
 Label:
 this code is vulnerable.

Now start to identify whether the following method contains a vulnerability or not.

Code:
 {Code}
 Label:

4.1.3 Implementations. As shown in Section 3.4, to extract diverse features, we employ ten groups of filters of different size, with dimension of $m \times 128 \times 6$, where m ranges from 1 to 10. Each filter size contains 32 filters to capture features from different regions of the input. We apply max pooling to convert the various features extracted by CNNs into feature values. Additionally, the calculation of centrality measures is shown in Table 1. Furthermore, the descriptions of other training hyperparameters are illustrated in Table 3. The whole model uses rectified linear units (*i.e.*, ReLU [24]) as the non-linear activation function. Moreover, the loss function used in our model to reward the correct classification and penalize the incorrect classification is cross-entropy loss. We train the model using Adam [49] with a learning rate of 0.0002. The training batch size on each machine is set to 32 for all experiments. We train *VulCNN* for 50 epochs on the training set and select the model that achieves the best performance on the validation set to validate the model’s ability on the test dataset. We report the average of these metrics across all ten validations to determine the final performance measure.

Four phases (*i.e.*, graph extraction, node embedding, image generation, and classification) in *VulCNN* are implemented by using Joern [93], sent2vec [70] and word2vec [74], networkx [10], and pytorch [11], respectively. For LLM-based baselines (*i.e.*, DeepSeek-Coder [38] and Llama-2-Instruct [85]), we conduct inference on a machine node with eight Nvidia A100-80G GPUs and 2T CPU memory. We run other experiments on a server with 32 cores of CPU and eight Nvidia 3090 GPUs.

4.1.4 Metrics. We adopt the widely used metrics to evaluate the detection results of *VulCNN* as shown in Table 4. The metrics used to measure the effectiveness of *VulCNN* are the same as others [56, 58, 102]. For example, *true positive* (TP) and *true negative* (TN) denote the number of samples correctly predicted as vulnerable and normal, respectively. We employ ten-fold cross-validation to assess the precision, recall, and F1 score for each validation. We then calculate the average of these metrics across all ten validations to determine the final performance measure.

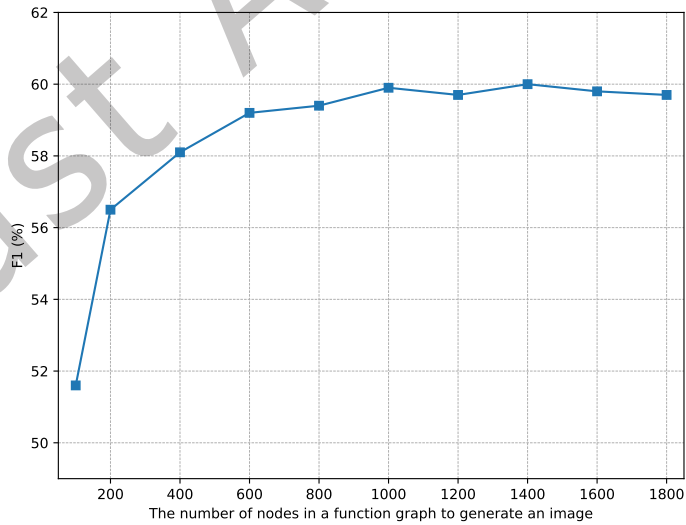
Table 3. Parameter settings in *VulCNN*

Parameters	Settings
Loss function	Cross Entropy Loss
Activation function	ReLU
Optimizer	Adam
Batch size	32
Learning rate	0.0002
Epoch num	50

Table 4. Descriptions of used metrics in our experiments

Metrics	Abbr	Definition
True Positive	TP	#samples correctly predicted as vulnerable
True Negative	TN	#samples correctly predicted as normal
False Positive	FP	#samples incorrectly classified as vulnerable
False Negative	FN	#samples incorrectly classified as normal
Accuracy	A	$(TP+TN)/(TP+TN+FP+FN)$
Recall	R	$TP/(TP+FN)$
Precision	P	$TP/(TP+FP)$
F-measure	F1	$2*P*R/(P+R)$

4.2 RQ1: Detection Performance

Fig. 8. The detection performance of *VulCNN* with different thresholds to construct images

In our method, we convert a function into multi-view images, so the height of the image (*i.e.*, the number of nodes to produce our images) is important for the performance of vulnerability detection. We first present the detection performance of *VulCNN* under different thresholds (*i.e.*, the image's height). Because more than 99% of the functions in Figure 6 have less than 2,000 nodes, we focus on the thresholds below 2,000 nodes. Specifically, we select a total of ten thresholds (*i.e.*, 100, 200, 400, 600, 800, 1,000, 1,200, 1,400, 1,600, and 1,800 nodes) to commence our evaluations. The experimental results are presented in Figure 8, through the results, we see that the threshold is positively correlated with detection accuracy, the larger the threshold, the higher the accuracy. On the other hand, the experimental results show that, when the threshold reaches 1,000 nodes, the growth of accuracy becomes small. Moreover, the greater the threshold, the larger the image, and the more memory required. Therefore, we select 1,000 nodes as our final threshold to generate the input multi-view images.

VulCNN achieves recall, precision, and F1 scores of 73.6%, 56.4%, and 63.8% on *Devign*, 58.6%, 38.8%, and 46.7% on *Big-Vul*, and 61.3%, 42.5%, and 50.2% on *ReVeal*, respectively. *VulCNN* demonstrates superior detection effectiveness across diverse benchmarks, achieving either the best or second-best results on most evaluation metrics. In particular, our improved *VulCNN* outperforms the previous version [91] on all three datasets, with consistent improvements in recall, precision, and F1 score.

4.2.1 Compared to Token and Slice-based Methods. Table 5 shows the detailed comparison results. For example, *TokenCNN* conducts lexical analysis to transform the source code into token sequences and embeds them into fixed-length vector representations, relying solely on surface-level lexical information for vulnerability detection. Its performance is relatively poor, achieving recall of 56.0%, precision of 42.9%, and F1 score of 48.6% on the *Devign* dataset. The precision is relatively low, indicating a high rate of false positives, and the recall is also not high, leading to a limited ability to detect real vulnerabilities. *VulDeePecker* and *SySeVR* enhance feature extraction by slicing programs into code gadgets and representing them as vectors. A *Bidirectional Recurrent Neural Network* (BRNN) is used to capture sequential dependency among code tokens. However, *VulDeePecker* achieves recall of 43.3%, precision of 51.5%, and F1 score of 47.1% on the *Devign* dataset, which is the lowest among these deep learning based approaches. *SySeVR* achieves recall of 44.1%, precision of 55.6%, and F1 score of 49.2% on the same dataset. Although *SySeVR* has a relatively higher precision compared to *VulDeePecker*, indicating fewer false positives, its low recall shows that many real vulnerabilities are missed. *VulDeeLocator* compiles source code into LLVM-IR intermediate representation and applies deep learning on the compiled features to detect vulnerabilities. It achieves recall of 55.4%, precision of 50.3%, and F1 score of 52.7% on the *Devign* dataset, outperforming *TokenCNN*, *VulDeePecker*, and *SySeVR*. Overall, these deep learning based approaches are limited because they either rely on surface lexical tokens or shallow program slices, which cannot sufficiently capture complex vulnerability patterns. Compared to these methods, *VulCNN* achieves significantly higher recall, precision, and F1 score on all evaluated datasets, demonstrating its advantage in both detecting more real vulnerabilities and maintaining fewer false alarms.

4.2.2 Compared to Graph-based Methods. Graph-based detection methods such as *Devign*, *EPVD*, and *VulDecgre* extract structural representations of programs to capture vulnerability patterns. *Devign* applies advanced program analysis to extract graph representations that encapsulate comprehensive program semantics. It achieves recall of 64.6%, precision of 51.2%, and F1 score of 57.2% on the *Devign* dataset, recall of 53.8%, precision of 33.2%, and F1 score of 41.0% on the *Big-Vul* dataset, and recall of 57.1%, precision of 26.3%, and F1 score of 36.0% on the *ReVeal* dataset. *EPVD* builds upon earlier graph-based methods by better leveraging program semantics to enhance detection performance. *VulDecgre* introduces an enhanced code graph structure that fuses multiple graphs and relational edges to better represent parameter dependencies and node type information, which are critical for capturing vulnerability patterns. This design enables *VulDecgre* to more accurately understand vulnerability behaviors and their potential impacts. It achieves recall of 71.6%, precision of 57.4%, and F1 score of 63.7% on the *Devign* dataset, recall of 57.3%, precision of 39.0%, and F1 score of 46.6% on the *Big-Vul* dataset, and recall

Table 5. Performance of our proposed method *VulCNN* and other baselines including *TokenCNN*, *VulDeePecker*, *SySeVR*, *VulDeeLocator*, *Devign*, *EPVD*, *VulDecgre*, *VulCNN (Old)*, *UniXcoder*, *PDBERT*, *Llama-2-Instruct*, *DeepSeek-Coder*, *Qwen2.5-Coder-Instruct*, and *GRACE* on detecting vulnerabilities. All values are in percentage (%).

Methods	Devign			Big-Vul			ReVeal		
	R	P	F1	R	P	F1	ReVeal	P	F1
<i>TokenCNN</i> [77]	56.0	42.9	48.6	54.2	24.8	34.0	40.6	16.2	23.2
<i>VulDeePecker</i> [103]	43.3	51.5	47.1	43.5	31.4	36.5	33.1	41.3	36.8
<i>SySeVR</i> [57]	44.1	55.6	49.2	45.4	37.5	41.1	33.8	40.3	36.7
<i>VulDeeLocator</i> [55]	55.4	50.3	52.7	48.6	29.3	36.6	48.8	22.9	31.1
<i>Devign</i> [102]	64.6	51.2	57.2	53.8	33.2	41.0	57.1	26.3	36.0
<i>EPVD</i> [98]	62.3	56.2	59.1	52.8	30.7	38.8	52.4	29.1	37.4
<i>VulDecgre</i> [43]	71.6	57.4	63.7	57.3	39.0	46.4	59.5	41.7	49.0
<i>VulCNN (Old)</i> [91]	69.2	54.6	61.1	54.3	37.0	44.0	56.4	37.2	44.8
<i>UniXcoder</i> [36]	57.3	61.4	59.3	53.8	35.7	42.9	55.1	45.8	50.0
<i>PDBERT</i> [62]	67.6	56.7	61.7	55.9	36.1	43.9	57.8	43.1	49.4
<i>Llama-2-Instruct</i> [13]	49.9	64.4	56.2	49.4	36.4	41.9	40.2	44.8	42.4
<i>DeepSeek-Coder</i> [38]	66.1	56.1	60.7	55.7	35.5	43.3	53.7	42.4	47.4
<i>Qwen2.5-Coder-Instruct</i> [41]	67.7	56.2	61.4	54.6	37.2	44.3	55.5	35.8	43.5
<i>Grace</i> [64]	73.0	55.8	63.3	58.3	38.3	46.2	60.4	41.5	49.2
<i>VulCNN</i>	73.6	56.4	63.8	58.6	38.8	46.7	61.3	42.5	50.2

of 59.5%, precision of 41.7%, and F1 score of 49.0% on the *ReVeal* dataset. These graph-based methods utilize features such as a function's control flow to detect vulnerabilities, achieving better results than methods that only use the source code for detection. However, these methods consider the nodes in a function graph to be equally important, ignoring the fact that some nodes are more likely to expose vulnerability behaviors. Compared to these graph-based methods, *VulCNN* achieves higher recall and F1 scores across all three datasets. Although the precision of *VulCNN* is slightly lower than that of *VulDecgre* on some datasets, *VulCNN* detects more real vulnerabilities overall, resulting in the best detection effectiveness among graph-based detection methods.

4.2.3 Compared to Pre-train-based Methods. *UniXcoder* and *PDBERT* leverage pre-training on large code corpora to capture semantic information. *UniXcoder* achieves recall of 57.3%, precision of 61.4%, and F1 score of 59.3% on the *Devign* dataset, while *PDBERT* achieves recall of 67.6%, precision of 56.7%, and F1 score of 61.7%. Although these methods achieve relatively high precision, their overall performance remains inferior to *VulCNN*. *VulCNN* maintains a better balance between detecting more real vulnerabilities and reducing false positives. Compared to these pre-trained methods, *VulCNN* achieves better vulnerability detection effectiveness by explicitly leveraging graph structural information during detection. While *PDBERT* attempts to enhance code understanding during pre-training, it does not explicitly incorporate function-level graph structures at inference time. *UniXcoder* processes code and AST in sequence form through mapping techniques, which inevitably lose some hierarchical structure and long-range relational information inherent to graphs. In contrast, *VulCNN* constructs multi-view function graphs and applies learnable centrality-aware weighting, allowing the model to capture global structural properties and focus on important nodes. By modeling rich relational information at detection time, *VulCNN* is

able to detect more real vulnerabilities while maintaining a relatively low false positive rate, thus achieving a better balance between recall and precision.

4.2.4 Compared to LLMs and LLM-based Methods. Among LLM-based approaches, *DeepSeek-Coder* and *Qwen2.5-Coder-Instruct* outperform *Llama-2-Instruct* but are still less effective than *VulCNN*. *DeepSeek-Coder* achieves recall of 66.1%, precision of 56.1%, and F1 score of 60.7% on the *Devign* dataset, while *Qwen2.5-Coder-Instruct* achieves recall of 67.7%, precision of 56.2%, and F1 score of 61.4%. Although *Qwen2.5-Coder-Instruct* achieves relatively high recall, its overall F1 score remains lower than *VulCNN*. While LLM-based models demonstrate strong language modeling capabilities, they require significantly higher computational resources. Deploying models such as *DeepSeek-Coder* and *Qwen2.5-Coder-Instruct* necessitates large GPU memory and leads to considerable inference time, yet their vulnerability detection effectiveness does not show a clear advantage over *VulCNN*. A detailed analysis of resource consumption and runtime overhead is provided in Section 4.4. For the LLM-based detection method, *GRACE* achieves a recall of 73.0%, a precision of 55.8%, and an F1 score of 63.3% on the *Devign* dataset. Although *GRACE* achieves competitive performance, its detection pipeline introduces considerable overhead by involving multiple complex steps, including graph construction, dimensionality reduction through T-SNE, semantic similarity matching, and final inference with LLMs. Also, the reliance on external LLM services (i.e., ChatGPT) incurs additional monetary cost. We further analyze the runtime overhead and financial cost associated with *GRACE* in Section 4.4.

In summary, *VulCNN* outperforms most of comparative methods and popular LLMs. It also achieves performance comparable to the state-of-the-art LLM-based vulnerability detection system. The superior performance of *VulCNN* can be attributed to its ability to integrate multiple types of program graphs (AST, CFG, and PDG) with six advanced centrality analysis methods. By utilizing AST, CFG, and PDG, *VulCNN* captures diverse aspects of program semantics, including syntactic structure, execution paths, and dependencies within the code. Besides, the incorporation of six centrality analysis methods—degree centrality, closeness centrality, betweenness centrality, eigenVector centrality, harmonic centrality, and Katz centrality—enhances the model’s ability to identify key components in the graph that contribute significantly to vulnerability detection. For example, eigenVector centrality highlights nodes connected to influential components, while Katz centrality considers both immediate and global influences within the graph. This comprehensive representation of program structure and behavior allows *VulCNN* to effectively identify vulnerabilities that other methods might overlook. When dealing with specific vulnerability functions, different centrality methods enable the classifier to prioritize various nodes from multiple perspectives. Additionally, learnable weights allow the model to focus on different graph representations of the samples, enhancing its detection capabilities. These combined innovations explain why *VulCNN* outperforms most popular vulnerability methods and achieves detection performance comparable to SOTA LLM-based approaches, while offering faster detection and incurring no token-related costs.

4.3 RQ2: Ablation Study

To evaluate the contributions of various components to the performance of *VulCNN*, we conducted a comprehensive ablation study. This study investigates the contribution of centrality metrics, the adaptive reweighting layer, and different function graph representations on the overall effectiveness of the model.

4.3.1 Contribution of Centrality Metrics. The contribution of centrality metrics to the effectiveness of *VulCNN* is evaluated through an ablation experiment in which the centrality metrics are removed from the model. Specifically, the centrality analysis is excluded by feeding the sentence vectors directly into the CNN model without multiplying them by the centrality values. The ablation experiment results are represented as *VulCNN-w/o centrality*, where the centrality metrics are excluded, and these results are compared to the full *VulCNN* model.

Table 6. Detection performance of *VulCNN* when specific components are removed, including centrality metrics and the adaptive re-weighting layer, as well as when using individual graph representations. All values are in percentage (%).

Methods	Devign			Big-Vul			ReVeal		
	R	P	F1	R	P	F1	R	P	F1
<i>VulCNN</i>	73.6	56.4	63.8	58.6	38.8	46.7	61.3	42.5	50.2
w/o centrality	67.3	53.9	59.9	54.3	38.3	44.9	57.9	39.1	46.7
w/o adaptive re-weighting layer	63.3	57.7	60.4	53.0	37.5	43.9	57.0	41.6	48.1
with only AST	62.8	55.2	58.8	51.8	36.5	42.8	56.1	36.8	44.5
with only CFG	70.5	53.8	61.0	55.1	34.5	42.5	55.7	41.8	47.8
with only PDG	69.6	56.7	62.5	55.8	38.3	45.4	59.6	41.0	48.6
with only CPG	69.8	57.1	62.8	56.5	38.5	45.8	60.2	41.6	49.2

Table 6 presents the recall, precision, and F1 scores of *VulCNN* and *VulCNN-w/o centrality* across three datasets: *Devign*, *Big-Vul*, and *ReVeal*. The results show that *VulCNN* achieves F1 scores of 63.8%, 46.7%, and 50.2% on the respective datasets, while *VulCNN-w/o centrality* achieves lower F1 scores of 59.9%, 44.9%, and 46.7%. Further examining the recall and precision metrics, removing centrality metrics causes the recall to drop from 73.6% to 67.3% on *Devign*, from 58.6% to 54.3% on *Big-Vul*, and from 61.3% to 57.9% on *ReVeal*. Similarly, the precision decreases from 56.4% to 53.9% on *Devign*, from 38.8% to 38.3% on *Big-Vul*, and from 42.5% to 39.1% on *ReVeal*. These results indicate that the centrality metrics not only help detect more real vulnerabilities, thereby improving recall, but also contribute to maintaining a lower false positive rate, as reflected in better precision. Unlike *VulCNN-w/o centrality* and other graph-based methods that treat all nodes as equally important, *VulCNN* leverages centrality metrics to prioritize more influential nodes in the graph. This allows the model to focus on critical regions of the code that are more likely to contribute to vulnerabilities, leading to a significant improvement in detection performance. By incorporating this nuanced understanding of node importance, *VulCNN* achieves better results compared to approaches that lack such differentiation.

Moreover, we further analyze the contribution of each centrality metric by individually removing it and observing the performance degradation across datasets. The results in Table 7 show that not all centrality measures contribute equally to the performance of *VulCNN*. Among the six centrality metrics, removing betweenness centrality leads to a larger performance decline on most metrics, with the F1 score dropping by 3.1% on *Devign*, 1.8% on *Big-Vul*, and 3.4% on *ReVeal*. This indicates that betweenness centrality, which quantifies the importance of nodes acting as critical intermediaries in control and data flow paths, is crucial for accurately detecting vulnerabilities, especially those involving control transfer or critical data flows. Eigenvector centrality also contributes significantly. Removing eigenvector centrality causes a 5.8% drop in recall and a 2.7% drop in F1 score on *Devign*. Eigenvector centrality favors nodes that are connected to other important nodes, allowing *VulCNN* to capture regions with layered influence. In vulnerability detection, such layered influential nodes often correspond to important control structures or frequently reused components that are prone to security risks.

Harmonic centrality similarly shows substantial contribution. Harmonic centrality better captures the global reachability of a node by considering inverse shortest-path distances. Its removal leads to consistent performance drops, suggesting that the ability to recognize nodes that are efficiently reachable across a function graph strengthens the understanding of code semantics and the identification of vulnerability propagation paths. Katz centrality and closeness centrality show moderate impacts. Katz centrality extends eigenvector centrality by considering influence propagation across longer paths with attenuation. While this helps in detecting vulnerabilities that are indirectly influenced by other code segments, its relative contribution is smaller compared to betweenness and

Table 7. The contribution of each centrality analysis to *VulCNN*. Darker colors indicate a greater decrease in detection performance when the corresponding centrality measure is removed. All values are in percentage (%).

Methods	Devign			Big-Vul			ReVeal		
	R	P	F1	R	P	F1	R	P	F1
<i>VulCNN</i>	73.6	56.4	63.8	58.6	38.8	46.7	61.3	42.5	50.2
w/o degree centrality	70.7 (-2.9)	56.3 (-0.1)	62.7 (-1.1)	57.8 (-0.8)	38.5 (-0.3)	46.2 (-0.5)	59.4 (-1.9)	41.1 (-1.4)	48.6 (-1.6)
w/o closeness centrality	70.1 (-3.5)	56.2 (-0.2)	62.4 (-1.4)	58 (-0.6)	38.3 (-0.6)	46.1 (-0.6)	60.3 (-1)	41.4 (-1.1)	49.1 (-1.1)
w/o harmonic centrality	68.1 (-5.5)	56.7 (0.3)	61.9 (-1.9)	57.3 (-1.3)	37.7 (-1.1)	45.5 (-1.2)	61.4 (0.1)	40.5 (-2)	48.8 (-1.4)
w/o betweenness centrality	68.1 (-5.5)	54.8 (-1.6)	60.7 (-3.1)	57.4 (-1.2)	36.8 (-2)	44.9 (-1.8)	57.6 (-3.7)	39.4 (-3.1)	46.8 (-3.4)
w/o eigenvector centrality	67.8 (-5.8)	55.5 (-0.9)	61.1 (-2.7)	55.5 (-3.1)	35.7 (-3.1)	43.5 (-3.2)	58.2 (-3.1)	39.7 (-2.8)	47.2 (-3)
w/o Katz centrality	69.8 (-3.8)	55.9 (-0.5)	62.1 (-1.7)	56.4 (-2.2)	36.5 (-2.3)	44.3 (-2.4)	60.1 (-1.2)	39.9 (-2.6)	47.9 (-2.3)

eigenvector centrality. Degree centrality causes the least performance degradation overall. It simply counts the number of immediate neighbors without considering their importance or role. In complex program graphs with rich semantics, this local information alone is insufficient to capture vulnerability-relevant patterns. However, in datasets like *Devign*, where the average number of nodes per sample is smaller (as shown in Figure 6), the local connectivity becomes relatively more important, and degree centrality still provides certain benefits. In simpler graphs, vulnerabilities are more likely associated with locally dense structures, making degree information modestly helpful.

Overall, these results demonstrate that different centrality metrics capture complementary structural properties of function graphs, such as local connectivity patterns, global influence propagation, and the identification of critical control points. By integrating multiple centrality measures, *VulCNN* is capable of balancing local and global structure awareness, significantly enhancing its vulnerability detection capabilities.

4.3.2 Contribution of the Adaptive Re-weighting Layer. The contribution of the adaptive re-weighting layer to the effectiveness of *VulCNN* is evaluated through an ablation experiment in which this layer is removed from the model. In this ablation setting, referred to as *VulCNN-w/o adaptive re-weighting layer*, all function graph representations (AST, CFG, and PDG) are assigned equal weights during integration, instead of dynamically learning their importance through the adaptive re-weighting layer. The experimental results in Table 6 show that removing the adaptive re-weighting layer leads to a noticeable decline in detection performance across all datasets. Specifically, the recall drops from 73.6% to 63.3% on *Devign*, from 58.6% to 53.0% on *Big-Vul*, and from 61.3% to 57.0% on *ReVeal*. Similarly, the precision decreases from 56.4% to 57.7% on *Devign*, from 38.8% to 37.5% on *Big-Vul*, and from 42.5% to 41.6% on *ReVeal*. While the precision change is moderate, the recall loss is more significant, indicating that without the adaptive re-weighting layer, the model misses more real vulnerabilities. The F1 scores decrease by 3.4%, 2.8%, and 2.1% on *Devign*, *Big-Vul*, and *ReVeal*, respectively, highlighting that

the adaptive re-weighting mechanism plays a crucial role in improving vulnerability detection by effectively integrating complementary information from multiple graph representations.

4.3.3 Contribution of Function Graph Representations. The contribution of individual function graph representations to the effectiveness of *VulCNN* was evaluated by conducting ablation experiments where only one type of graph representation (AST, CFG, or PDG) was used at a time. The results of these experiments are presented in Table 6. We further analyze the importance of individual function graph representations by using only one type of graph at a time. Among these settings, using only the AST yields the worst performance. The recall drops to 62.8%, the precision to 55.2%, and the F1 score to 58.8% on the *Devign* dataset. This substantial reduction shows that the AST alone, which primarily captures syntactic structures, lacks sufficient semantic or dependency information necessary for precise vulnerability detection. Using only the CFG slightly improves the results compared to AST. With CFG alone, the recall reaches 70.5%, the precision is 53.8%, and the F1 score is 61.0% on the *Devign* dataset. While the recall remains relatively high, the overall performance is still inferior to the full model, reflecting the limitations of focusing solely on control flow without capturing data dependencies. Among the three, using only the PDG achieves the best results. When using only PDG, the recall reaches 69.6%, the precision rises to 56.7%, and the F1 score improves to 62.5% on the *Devign* dataset. This demonstrates that PDG, by capturing both control flow and data flow information, provides richer semantic context and is more effective for vulnerability detection than AST or CFG alone. The analysis of the results reveals that each graph representation provides distinct and valuable information to *VulCNN*.

The combination of AST, CFG, and PDG representations outperforms any single representation. This is because the integration of multiple graph types offers complementary perspectives on the code. While AST provides syntactic information, CFG focuses on control flow, and PDG delivers a richer representation by combining control and data flow. The adaptive re-weighting layer further enhances this combination by dynamically learning the importance of each representation, allowing the model to effectively utilize the strengths of each graph type. This synergy between diverse graph representations and the adaptive re-weighting mechanism leads to superior vulnerability detection performance compared to using any single representation alone.

Additionally, *VulCNN* converts the AST, CFG, and PDG of each function into separate images, extracts features using CNNs, and fuses them through an adaptive re-weighting layer. Alternatively, it is possible to perform feature fusion before CNN-based feature extraction, specifically by using a hybrid graph such as the CPG to represent the function and then applying CNNs directly for vulnerability detection. Compared to *VulCNN*, this approach eliminates the need to generate multi-view graphs or use an adaptive re-weighting layer. To further validate the effectiveness of *VulCNN*, we compare the performance of these two methods.

Specifically, we use Joern to process the same dataset and generate the CPG for each function. Each function is then converted into a single six-channel image, from which CNNs extract features for classification using a fully connected layer. This experimental setup is consistent with the ablation experiments, where only a single function graph (i.e., AST, CFG, or PDG) is used. Table 6 presents the performance comparison between the two approaches. Using only the hybrid CPG graph slightly outperforms the PDG-only method on the *Devign*, *Big-Vul*, and *ReVeal* datasets, and also surpasses the results achieved by using only AST or CFG. This indicates that hybrid graphs capture more comprehensive semantic information of functions, which helps CNNs extract more informative features. However, this approach still underperforms compared to *VulCNN*. There are two possible reasons for this gap. First, although hybrid graphs integrate more structural information, they tend to be more complex, making it harder for centrality metrics to identify structurally important nodes. Second, CNNs may struggle to extract effective representations from these densely fused graphs, reducing the overall quality of the learned features for classification.

In contrast, the adaptive re-weighting layer in *VulCNN* allows for a more flexible combination of label-relevant features from each perspective, which contributes to better detection performance. This further demonstrates

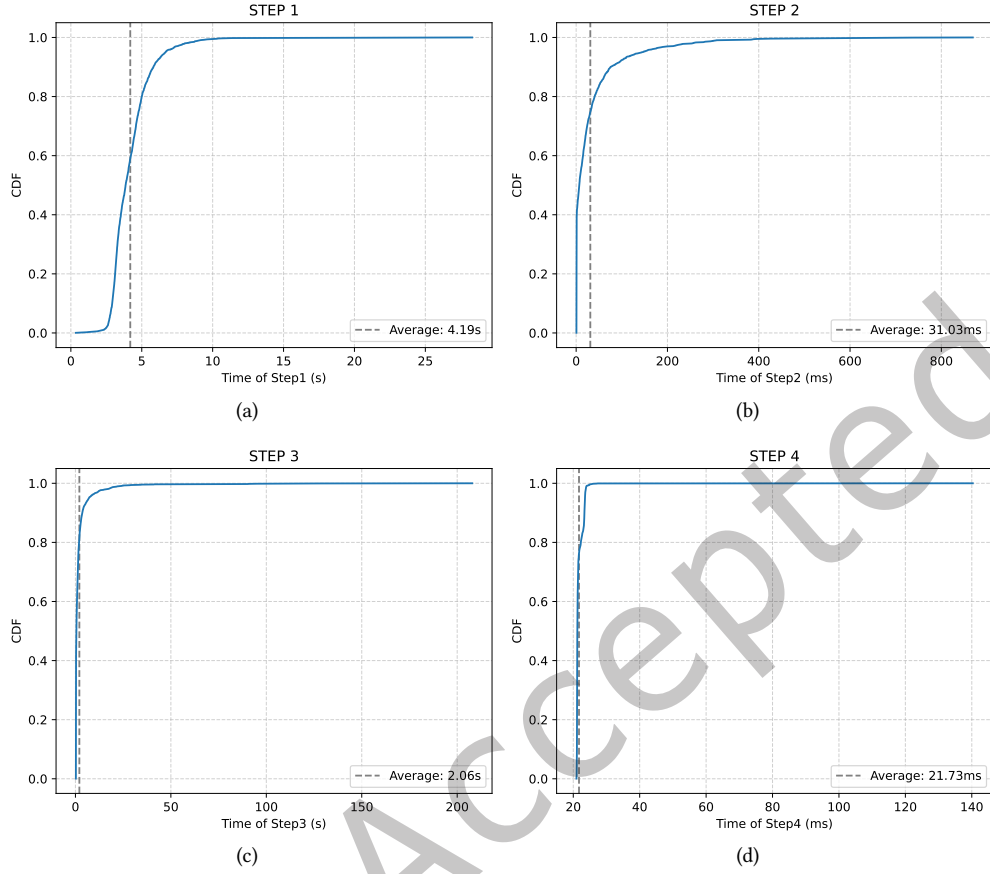


Fig. 9. The *Cumulative Distribution Function* (CDF) of runtime overheads of *VulCNN* on different phases

that generating multi-view images from different perspectives facilitates more effective centrality analysis of node properties and enables CNNs to extract features more easily. The adaptive re-weighting layer also enables more flexible feature fusion. All these factors contribute significantly to the effectiveness of *VulCNN*.

4.4 RQ3: Runtime Overhead

In this section, we perform a comprehensive evaluation on the runtime overhead of *VulCNN*. Given a new function, *VulCNN* consists of four main steps to complete the classification: *Graph Extraction*, *Node Embedding*, *Image Generation*, and *Classification*.

4.4.1 Graph Extraction. Given the source code of a function, the first step of *VulCNN* is to extract the AST, CFG, and PDG of it. Figure 9 presents the runtime overheads of graph extraction on our merged dataset. Over 90% of the functions have their graph representations extracted in less than six seconds. On average, it takes 4.19 seconds to construct three graph representations. Since we can utilize *joern* [93] to process function graphs in parallel, the processing time does not significantly increase when compared to generating a single graph representation.

4.4.2 Node Embedding. The second step of *VulCNN* involves embedding all nodes in the graph representations into corresponding fixed-length vectors. We use *sent2vec* to embed the nodes in the PDG and *word2vec* to embed the nodes in the AST and CFG. Converting node text to embeddings is a fast process. As shown in Figure 9, it takes only about 31.03 milliseconds on average to complete the text embedding of three graph representations.

4.4.3 Image Generation. After embedding all nodes into vector representations, the third step of *VulCNN* is to apply centrality analysis to transform the new graphs into multi-view images. Figure 9 presents the runtime overheads of *VulCNN* during this step. Despite some complex functions having graphical representations with many nodes, the centrality analysis method remains efficient. We observed that more than 90% of graphs can be converted into images within four seconds. Due to parallel processing by the CPUs, the overall time consumed in this third stage is approximately 2.06 seconds on average. These experimental results indicate that *VulCNN* can efficiently transform graph representations into multi-view images.

4.4.4 Classification. The final step of *VulCNN* involves detecting vulnerabilities using a trained model, which contains a parameter-shared CNN and three learnable weights. The runtime overhead of *VulCNN* during the classification phase is shown in Figure 9. We found that CNN classification is the fastest phase in *VulCNN*, taking only approximately 0.02 seconds to complete the classification of multi-view images (i.e., three images with six channels). Additionally, for a single image, the time consumption for classification is about 0.016 seconds. This indicates that additional images do not proportionally increase time consumption, thanks to the parallelization of matrix operations in GPUs.

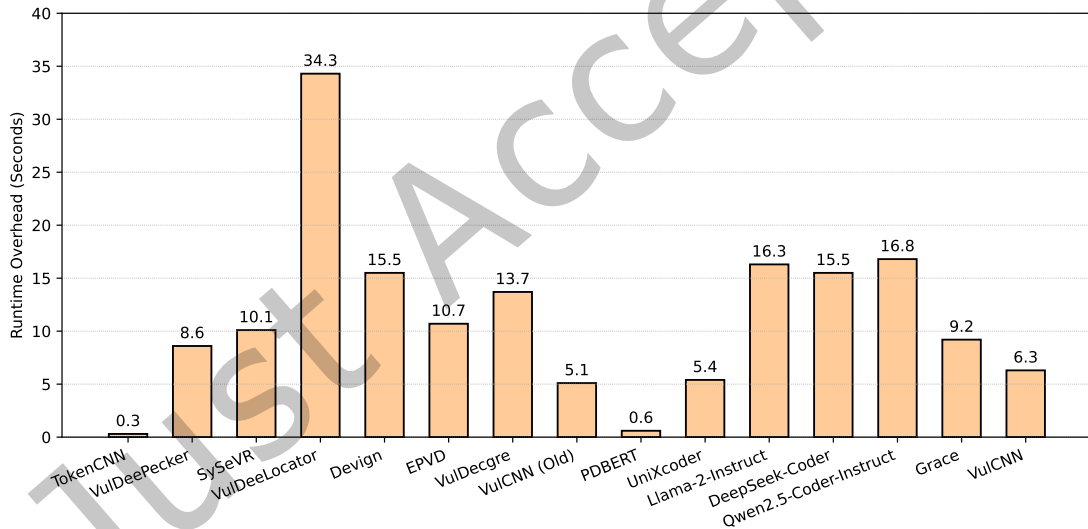


Fig. 10. Comparative runtime overheads of *TokenCNN*, *VulDeepPecker*, *SySeVR*, *VulDeeLocator*, *Devign*, *EPVD*, *VulDecgre*, *VulCNN (Old)*, *PDBERT*, *UniXcoder*, *Llama-2-Instruct 7B*, *DeepSeek-Coder 6.7B*, *Qwen2.5-Coder-Instruct 7B*, *GRACE*, and *VulCNN*.

In addition, we also compare the scalability of *VulCNN* with other comparative systems. We compute the inference time consumption of baselines and our proposed method on the test set of the merged dataset. Figure 10 presents the average runtime overheads of *TokenCNN*, *PDBERT*, *UniXcoder*, *VulDeepPecker*, *SySeVR*, *VulDeeLocator*, *Devign*, *EPVD*, *VulDecgre*, *Llama-2-Instruct 7B*, *DeepSeek-Coder 6.7B*, *Qwen2.5-Coder-Instruct 7B*, *GRACE*, and *VulCNN*.

As for *TokenCNN*, because it only applies simple lexical analysis to obtain the source code tokens and uses a CNN model to detect vulnerability, it is the fastest. On average, it only takes 0.3 seconds to finish the analysis of a function in the merged dataset. However, *VulCNN* only considers the source code as text and employs a CNN model to detect vulnerabilities. It does not incorporate other semantic representations of the code, such as function graphs. Consequently, its detection performance has a significant gap compared to other methods that utilize these additional semantic representations. As for *VulDeePecker* and *SySeVR*, their slice generation phases also extract the function graph representations such as the *control flow graph* (CFG), *program dependency graph* (PDG), and *function call graph* (FCG), resulting in a similar inference time consumption with *VulCNN* in the extract phase. In contrast, these methods utilize LSTM as the classifier to detect vulnerabilities, which is a type of Recurrent Neural Network (RNN) for processing sequential data, and the computation of each time step depends on the output of the previous time step. Therefore, the runtime overhead of these two methods is larger than that of *VulCNN*. Moreover, our method also achieves better performance than these two methods due to our centrality analysis and learnable adaptable weights. As for *VulDeeLocator*, as it is an LLVM-IR-based vulnerability detection system, the source code needs to be compiled first. The compilation phase consumes lots of time, making it difficult to detect vulnerabilities on a large scale.

As for *Devign*, it combines different code representations (*i.e.*, AST, CFG, PDG, and *natural code sequence* (NSC)) to achieve accurate vulnerability analysis. However, it utilizes the graph neural networks (GNN) to learn nodes related to the vulnerabilities. CNN is generally faster than GNN. This is because CNNs can leverage the regular structure of image data to enable highly parallel computation. In contrast, the graph structure data processed by GNNs is more complex and irregular, leading to lower computational efficiency. Therefore, *Devign* consumes more runtime than *VulCNN*. For *EPVD*, it extracts the AST and CFG, and obtains multiple execution paths from an entry node to its exit node. Then *EPVD* utilizes the pre-trained code language model (*i.e.*, CodeBERT [33]) and CNN to detect vulnerabilities. Its runtime overhead is larger than that of our approach, due to the introduction of additional pre-trained models. It also requires more memory to deploy than *VulCNN*. For the pre-trained code representation methods, *PDBERT* and *UniXcoder*, although they achieve faster inference speeds than *VulCNN*, 0.6 and 5.4 seconds, their detection performance is worse than ours. Besides, the *PDBERT* and *UniXcoder* models are larger, with about 125M and 126M parameters, respectively, leading to higher GPU memory consumption compared to our lightweight *VulCNN*. Compared to the previous version, *VulCNN* increases the average detection time per function by 1.2 seconds but achieves consistently improved detection performance across all datasets.

For LLM methods, we employ them using eight 3090 GPUs and utilize FP16 to save GPU memory. Llama-2-Instruct and DeepSeek-Coder take 16.3 seconds and 15.5 seconds on average to finish detecting a function, respectively. Their runtime overhead is higher than that of *VulCNN*, due to the number of LLM parameters being much larger than that of traditional DL-based detection methods. Moreover, deploying LLMs consumes significantly more GPU memory. For instance, deploying DeepSeek-Coder [38] with 6.7 billion parameters requires approximately 16GB of GPU memory, whereas *VulCNN* requires only about 0.3 GB. As for *Qwen2.5-Coder-Instruct*, its detection effectiveness is relatively poor, and its inference time is also high at 16.8 seconds, making it slower and less efficient. For the LLM-based method *Grace*, we evaluate it on only 2,000 functions (randomly choosing 1,000 vulnerable functions and 1,000 normal functions) due to the high cost associated with API calls. *Grace* integrates ChatGPT to assist in detection. Its inference time reaches 9.2 seconds per function, and it requires calling external API services, which cause a significant monetary cost, making large-scale vulnerability detection extremely expensive. Also, since ChatGPT is a closed-source model, it is difficult to obtain internal logits, resulting in limited interpretability. Although the performance of *Grace* is comparable to ours, our method is faster, more economical, and more controllable.

In summary, *VulCNN* achieves faster inference speed than most existing vulnerability detection methods, including graph-based methods such as *VulDecgre* and LLM methods including *Llama-2-Instruct*, *DeepSeek-Coder*, and *Qwen2.5-Coder-Instruct*. Although the inference speed of *VulCNN* is slightly slower than pre-trained code

Table 8. Detection performance of *VulCNN* by using pre-trained code embedding models. All values are in percentage (%).

Embedding Methods	Devign			Big-Vul			ReVeal		
	R	P	F1	R	P	F1	R	P	F1
<i>VulCNN</i>	73.6	56.4	63.8	58.6	38.8	46.7	61.3	42.5	50.2
<i>CodeBERT</i> [33]	61.1	57.9	59.4	54.2	33.8	41.6	53.2	31.1	39.3
<i>VulCNN_{CodeBERT}</i>	68.9	59.3	63.7	56.2	39.0	46.1	60.9	42.2	49.8
<i>GraphCodeBERT</i> [37]	66.9	55.8	60.9	57.7	38.3	46.0	58.1	40.4	47.6
<i>VulCNN_{GraphCodeBERT}</i>	72.6	58.1	64.5	59.9	38.6	47.0	60.3	42.7	50.0

representation methods such as *PDBERT* and *UniXcoder*, it achieves better vulnerability detection performance than these methods. Particularly, *VulCNN* achieves slightly better vulnerability detection effectiveness than the state-of-the-art method *GRACE*, while reducing the time overhead by 31.52% (calculated as $(9.2 - 6.3)/9.2$).

4.5 RQ4: Performance and Efficiency Comparison of Different Code Embedding Methods

VulCNN uses traditional code embedding methods (*i.e.*, Word2Vec and Sent2Vec) to convert nodes in the function graph into embeddings. Recently, pre-trained code embedding methods have shown the ability to capture richer semantic representations of source code, which may improve performance on downstream tasks. To evaluate whether such pre-trained embeddings can enhance the performance of *VulCNN*, we replace Word2Vec and Sent2Vec with two pre-trained code embedding models: CodeBERT [33] and GraphCodeBERT [37]. Both models output embeddings with a hidden size of 768, which is substantially larger than the 128-dimensional embeddings used in our original setup. We apply a fully connected layer to project the 768-dimensional outputs to 128 dimensions to ensure a fair comparison. The parameters of this layer are optimized during training. All other experimental settings remain consistent with those of *VulCNN*.

Table 8 presents the vulnerability detection performance of *VulCNN* when using pre-trained code embedding models. First, we observe that *VulCNN* with integrated embeddings significantly outperforms the standalone pre-trained models. This result is expected, as *VulCNN* leverages additional structural information from function graphs, and incorporates a centrality analysis module, a feature extraction module, and an adaptive re-weighting layer. On the other hand, replacing traditional embeddings with CodeBERT does not lead to performance improvements. This may be attributed to the fact that CodeBERT is pre-trained solely on raw source code without incorporating structural or graph-based information [33]. As a result, it does not offer clear advantages for node embedding within function graphs. The general-purpose knowledge captured during pre-training may introduce noise or mismatched representations that negatively impact performance in this context. GraphCodeBERT slightly improves *VulCNN*, with F1 score increases of less than 1%. Compared to CodeBERT, GraphCodeBERT incorporates function graph and data flow information during pre-training, which benefits node embeddings [37]. However, it is important to note that this improvement comes at a higher computational cost. Traditional embedding methods function similarly to a lookup table and can produce embeddings on the CPU within milliseconds. In contrast, GraphCodeBERT requires GPU inference, consumes more memory, and introduces longer processing times, which may limit its scalability in practice.

In summary, incorporating a pre-trained code embedding model such as GraphCodeBERT can slightly enhance the detection performance of *VulCNN*, but at the cost of increased GPU resource usage, greater memory consumption, and slower inference. Both pre-trained code embedding models and traditional embedding methods

present distinct trade-offs that should be considered when designing scalable and effective vulnerability detection systems.

5 DISCUSSION

5.1 Practicability

To evaluate the practicability of *VulCNN*, we conduct an extensive experiment utilizing the National Vulnerability Database (NVD). We collect a total of 169 CVEs spanning from May 2024 to May 2025, which represent a diverse range of real-world software security vulnerabilities. Our *VulCNN* tool is then deployed to scan the functions associated with these vulnerabilities. The results of our experiment are compelling. *VulCNN* successfully detects 126 out of the 169 CVEs, showcasing its effectiveness in identifying real-world security issues. We also run our comparative vulnerability detection tools against the same dataset for comparison. Among them, *GRACE* detects 118 CVEs, the highest number among all the compared tools except *VulCNN*. However, *VulCNN* outperforms *GRACE* by detecting more vulnerabilities, clearly validating its superior performance and practical viability in real-world vulnerability detection scenarios.

5.2 Threats to Validity

During the training phase of the classification process, the CNN requires input images of equal size. However, since the number of nodes in function graphs varies, a fixed threshold is necessary to generate images of consistent size. The selection of a maximum node number for these function graphs to create equal-size images may introduce inaccuracies. Additionally, padding images with zeros to achieve a uniform size might impact detection performance. To address this threat, we analyzed the node numbers of over 53,000 functions across three distinct datasets to determine a better threshold. Furthermore, we conducted experiments to demonstrate that zero-padding does not significantly impair detection efficiency (as evidenced in Figure 8). These findings confirm the applicability and robustness of our chosen thresholds, and demonstrate that *VulCNN* remains effective across a range of thresholds.

Another threat arises from one of the primary challenges faced by general vulnerability detection systems, *i.e.*, model generalization [98, 102]. To evaluate the generalization capability of *VulCNN*, we trained the model on two widely used datasets (*i.e.*, *Big-Vul* and *Devign*), and incorporated the *ReVeal* dataset exclusively into the test set. The results indicate that *VulCNN* achieves competitive detection performance on the *ReVeal* dataset, despite the absence of *ReVeal* training data. This finding suggests that the proposed strategy of generating multi-view images and re-weighting-based classification demonstrates a better generalization.

Recently, large language models (LLMs) have demonstrated impressive performance in vulnerability detection. To comprehensively compare LLMs with our proposed method, we conducted extensive experiments on three datasets using two widely adopted LLMs (*i.e.*, *Llama-2-Instruct* and *DeepSeek-Coder*). The experimental results reveal that *VulCNN* outperforms LLM-based methods in vulnerability detection. Additionally, we observed that LLM-based approaches require significantly more GPU memory and longer inference time. These findings indicate that *VulCNN* not only exhibits superior detection capability but also operates more efficiently compared to current LLM-based methods. Moreover, our method also achieves better performance compared to SOTA DL-based detection systems (*i.e.*, *Devign* and *EPVD*).

5.3 Future work

While working on *VulCNN*, several insightful considerations arose. To our knowledge, we may enhance *VulCNN* or develop novel approaches to detect vulnerabilities from four perspectives:

- Motivated by the adaptive re-weighting layer of *VulCNN*, we can further explore the development of an efficient vulnerability detection framework. We know that different graph representations contribute

variably to vulnerability detection. For instance, some vulnerabilities may be detectable with a simple AST, while others necessitate more comprehensive information from representations such as CFG and PDG. Based on this understanding, we can develop an efficient detection framework that selects the most appropriate representation for each function, thereby providing interpretability and resource efficiency. This framework would aim to use the minimum necessary information to effectively detect vulnerabilities, rather than processing the entire codebase, thereby conserving computational resources.

- *VulCNN* proposes a novel and efficient method of converting code into image, so we may achieve a better detection vulnerability with the help of today's impressive MLLM. Specifically, we can utilize the images generated by *VulCNN* to align with the textual descriptions of the function's ground truth, thereby fine-tuning an MLLM based on such alignments. By doing so, we can perform vulnerability detection directly from the images using the fine-tuned MLLM. Furthermore, the images can also be aligned with explanations of the vulnerabilities, enabling us to generate detailed explanations of the vulnerabilities using the function's image and an MLLM. This alignment may achieve a vulnerability detection system with better interpretability.
- From the results in Figure 9, we see that the most time-consuming phase of *VulCNN* is to extract the graph representations of functions, this phase occupies more than 87% of the total processing runtime. In our future work, we plan to design a new static analysis tool or try other static analysis tools (e.g., *Frama-C* [7]) to achieve more efficient function graph generation.
- In *VulCNN*, we select six centrality, i.e., degree centrality, katz centrality, closeness centrality, harmonic centrality betweenness centrality, and eigenVector centrality to commence our image transformation. In practice, various centrality analysis methods may contribute differently to the detection of vulnerabilities. In our approach, we treat each centrality analysis as a separate channel of the image. However, there is potential for further optimization. We aim to identify an efficient combination method for different centrality methods at the sample level, which could enhance the efficacy of *VulCNN*.
- We currently use a traditional convolutional neural network backbone in *VulCNN*, but future work can leverage more recent architectures. Vision Transformers, such as ViT [27], apply self-attention over the entire code image to capture global patterns and improve interpretability. Hybrid CNN-Transformer models like CoAtNet [25] combine convolutional layers for local feature extraction with transformer layers for global context, offering a balanced approach to accuracy and efficiency. EfficientNet [83] scales network depth and width to deliver high accuracy at a lower computational cost, which is useful for large codebases. ConvNeXt [61] incorporates design ideas from transformers into a standard CNN, providing a simple alternative with strong performance. Evaluating these models on our code-image dataset will help us find the best trade-off between detection accuracy, inference speed, and resource use.

6 RELATED WORK

There have been proposed many different vulnerability detection methods, which can be classified into two main categories: code-similarity-based and pattern-based.

As for similarity-based methods, the similarity can be measured from different aspects, such as string-based [42, 48], tree-based [44, 71], token-based [46, 78], graph-based [53], and their hybrid-based [56]. However, they can only detect cloned vulnerabilities and cannot detect new vulnerabilities [58]. To tackle this challenge, pattern-based techniques have been designed.

According to the degree of automation, pattern-based work can be divided into three subcategories (i.e., manual methods, semi-automatic methods, and automatic methods).

1) Manual methods: Human experts manually generate vulnerability patterns and use them to detect new vulnerabilities. In practice, the detection effectiveness of these tools (e.g., *Checkmarx* [5], *FlawFinder* [6], and

RATS [9]) is poor since experts cannot generate all patterns of different vulnerabilities. Results in Figure ?? also demonstrate the situation.

2) Semi-automatic methods [16, 79, 80, 95]: Human experts first extract certain features (e.g., subtrees and API symbols [94], imports and function calls [68]) and then feed them to traditional machine learning models (e.g., support vector machine and k-nearest neighbor) to detect vulnerabilities.

3) More automatic methods (i.e., deep-learning-based methods): Since deep learning can automatically extract features from source code, it has been used extensively for source code vulnerability detection [22, 30, 57–59, 77, 102, 103]. For example, *VulDeePecker* [58] first collects code gadgets by slicing programs and then transforms them into corresponding vector representations. Finally, it uses these vectors to train a *Bidirectional Long Short Term Memory* (BLSTM) model to detect vulnerabilities. *muVulDeePecker* [103] uses the program processing method in *VulDeePecker* and adds a code attention mechanism to detect multi-class vulnerabilities.

Devign [102] applies a general graph neural network (GNN) to detect vulnerabilities. It contains a novel convolutional module that can effectively extract useful features from rich node representations for graph-level classification. *DeepWukong* [22] distills program semantics into program dependency graphs and splits them into several subgraphs according to points of interest, feeding these subgraphs into a GNN for vulnerability detection. *AMPLE* [89] combines graph simplification with enhanced graph representation learning to improve vulnerability detection. *ReGVD* [69] encodes source code into a graph structure where nodes correspond to code tokens initialized using pre-trained code models. *EPVD* [98] segments code into distinct execution paths derived from the CFG, learns unique representations for each path, and leverages pre-trained code models to achieve state-of-the-art vulnerability detection results. Additionally, recent works such as *VulDecGRE* [43] and *MGVD* [73] have further improved graph-based vulnerability detection. *VulDecGRE* strengthens graph representations by explicitly modeling global dependency relationships, enabling more accurate vulnerability identification. *MGVD* leverages multiple complementary graph structures to capture diverse code semantics, leading to better detection performance across various vulnerability types. *Concoction* [40] introduces a deep learning framework that combines static code representations with dynamic execution traces to enhance vulnerability detection. By employing unsupervised active learning techniques, it identifies critical execution paths for symbolic analysis, thereby reducing overhead while capturing deep program semantics. *ContraFlow* [23] presents a path-sensitive code embedding approach utilizing self-supervised contrastive learning. It focuses on preserving feasible value-flow paths through a pretrained path embedding model, enabling precise vulnerability detection with reduced reliance on large labeled datasets.

Meanwhile, pre-trained code representation models have also achieved notable improvements. For example, *UniXcoder* [36] introduces a unified cross-modal pre-training framework designed for code understanding tasks. *CodeT5+* [88] proposes an enhanced encoder-decoder architecture optimized for programming languages, showing strong results across different code-related benchmarks. In addition, PDBERT [62] explicitly pre-train models by predicting program dependencies, demonstrating further gains in vulnerability analysis using transformer-based models.

Large Language Models (LLMs) have been extensively utilized across natural language processing tasks [13, 26, 54, 76, 85, 99]. Their capabilities have garnered significant interest in software engineering tasks such as coding, debugging, and refactoring [67, 86]. Several recent studies propose leveraging LLMs for vulnerability detection [35, 45, 65, 96, 97]. *DLAP* [97] introduces a prompting framework combining deep learning models and LLMs. These methods fix LLM parameters and leverage in-context learning [84] to improve vulnerability detection effectiveness. Others enhance LLM vulnerability detection capabilities through supervised fine-tuning on instruction tuning datasets [13, 38, 63, 76]. Notably, *GRACE* [64] enhances LLM-based vulnerability detection by integrating graph structural information and in-context learning. Specifically, it transforms code property graphs into natural language descriptions to provide structural context and employs a demonstration retrieval method that selects relevant code examples based on semantic, lexical, and syntactic similarities. This combined

approach significantly improves the performance of LLMs in detecting software vulnerabilities. Further extending this direction, specialized LLMs for coding, such as *Qwen2.5-Coder-7B-Instruct* [41], exemplify the cutting-edge capability of foundation models adapted specifically for software vulnerability detection.

7 CONCLUSION

In this paper, we introduce an innovative approach that efficiently transforms the source code of a function into multi-view images, preserving diverse program information from multiple perspectives. Building on this idea, we propose a scalable and efficient graph-based vulnerability detection system, *VulCNN*, which leverages function images and convolutional neural networks to detect vulnerabilities effectively. The experimental evaluation, conducted on three widely used datasets (*i.e.*, the *Devign*, *Big-Vul*, and *ReVeal* datasets), demonstrates that *VulCNN* outperforms nine state-of-the-art vulnerability detectors (*i.e.*, *TokenCNN* [77], *VulDeePecker* [58], *SySeVR* [57], *VulDeeLocator* [55], *Devign* [102], *EPVD* [98], *VulDecgre* [43], *UniXcoder* [36], and *PDBERT* [62]). Moreover, a comparison with three widely used LLMs (*i.e.*, *Llama-2-Instruct* [85], *DeepSeek-Coder* [38], and *Qwen2.5-Coder-Instruct* [41]) as well as one state-of-the-art LLM-based detection system (*i.e.*, *GRACE* [64]) further shows its effectiveness and efficiency. *VulCNN* is an effective and practical solution for large-scale vulnerability detection, achieving a balance between scalability and detection accuracy, making it a valuable contribution to secure software development.

REFERENCES

- [1] 2020. 5 key takeaways from the 2020 Open Source Security and Risk Analysis report. <https://securityboulevard.com/2020/05/5-key-takeaways-from-the-2020-open-source-security-and-risk-analysis-report>.
- [2] 2020. The Exactis Breach: 5 Things You Need to Know. <https://blog.infoarmor.com/individuals-and-families/the-exactis-breach-5-things-you-need-to-know>.
- [3] 2020. WannaCry ransomware attack. https://en.wikipedia.org/wiki/WannaCry_ransomware_attack.
- [4] 2021. Adjacency Matrix. https://en.wikipedia.org/wiki/Adjacency_matrix/.
- [5] 2021. Checkmarx. <https://www.checkmarx.com/>.
- [6] 2021. FlawFinder. <http://www.dwheeler.com/flawfinder/>.
- [7] 2021. Frama-C. <http://frama-c.com/>.
- [8] 2021. Open-source code analysis platform for C/C++ based on code property graphs. <https://joern.io/>.
- [9] 2021. Rough Audit Tool for Security. <https://code.google.com/archive/p/rough-auditing-tool-for-security/>.
- [10] 2021. Software for complex networks (Networkx). <http://networkx.github.io>.
- [11] 2021. Tensors and Dynamic neural networks in Python with strong GPU acceleration (PyTorch). <https://pytorch.org>.
- [12] 2023. FFmpeg. <http://www.ffmpeg.org/>.
- [13] 2023. Llama-2. <https://ai.meta.com/research/publications/llama-2-open-foundation-and-fine-tuned-chat-models>.
- [14] 2023. Qemu. <https://www.qemu.org/>.
- [15] Essa A Alhazami and Abdullah M Sheneamer. 2023. Graph-of-code: Semantic clone detection using graph fingerprints. *IEEE Transactions on Software Engineering* 49, 8 (2023), 3972–3988.
- [16] Michael Backes, Boris Köpf, and Andrey Rybalchenko. 2009. Automatic discovery and quantification of information leaks. In *Proceedings of the 2009 IEEE Symposium on Security and Privacy (S&P'09)*. 141–153.
- [17] Phillip Bonacich. 2007. Some unique properties of eigenvector centrality. *Social networks* 29, 4 (2007), 555–564.
- [18] Ulrik Brandes. 2001. A faster algorithm for betweenness centrality. *Journal of mathematical sociology* 25, 2 (2001), 163–177.
- [19] Xiansheng Cao, Junfeng Wang, Peng Wu, and Zhiyang Fang. 2024. VulMPFF: A Vulnerability Detection Method for Fusing Code Features in Multiple Perspectives. *IET Information Security* 2024, 1 (2024), 4313185.
- [20] Saikat Chakraborty, Rahul Krishna, Yangruibo Ding, and Baishakhi Ray. 2020. Deep Learning based Vulnerability Detection: Are We There Yet? *IEEE Transactions on Software Engineering* abs/2009.07235 (2020).
- [21] Saikat Chakraborty, Rahul Krishna, Yangruibo Ding, and Baishakhi Ray. 2022. Deep Learning Based Vulnerability Detection: Are We There Yet? *IEEE Transactions on Software Engineering* 48, 9 (2022), 3280–3296. <https://doi.org/10.1109/TSE.2021.3087402>
- [22] Xiao Cheng, Haoyu Wang, Jiayi Hua, Guoai Xu, and Yulei Sui. 2021. Deepwukong: Statically detecting software vulnerabilities using deep graph neural network. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 30, 3 (2021), 1–33.
- [23] Xiao Cheng, Guanqin Zhang, Haoyu Wang, and Yulei Sui. 2022. Path-sensitive code embedding via contrastive learning for software vulnerability detection (*ISSTA 2022*). Association for Computing Machinery, New York, NY, USA, 519–531. <https://doi.org/10.1145/>

- 3533767.3534371
- [24] George E. Dahl, Tara N. Sainath, and Geoffrey E. Hinton. 2013. Improving deep neural networks for LVCSR using rectified linear units and dropout. In *Proceedings of the 2013 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP'13)*. 8609–8613.
 - [25] Zihang Dai and et al. 2021. CoAtNet: Marrying Convolution and Attention for All Data Sizes. In *Advances in Neural Information Processing Systems*.
 - [26] DeepSeek-AI. 2024. DeepSeek LLM: Scaling Open-Source Language Models with Longtermism. *arXiv preprint arXiv:2401.02954* (2024). <https://github.com/deepseek-ai/DeepSeek-LLM>
 - [27] Alexey Dosovitskiy and et al. 2020. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *arXiv preprint arXiv:2010.11929* (2020).
 - [28] Shihan Dou, Yan Liu, Haoxiang Jia, Limao Xiong, Enyu Zhou, Junjie Shan, Caishuang Huang, Wei Shen, Xiaoran Fan, Zhiheng Xi, et al. 2024. StepCoder: Improve Code Generation with Reinforcement Learning from Compiler Feedback. *arXiv preprint arXiv:2402.01391* (2024).
 - [29] Xiaohu Du, Ming Wen, Jiahao Zhu, Zifan Xie, Bin Ji, Huijun Liu, Xuanhua Shi, and Hai Jin. 2024. Generalization-Enhanced Code Vulnerability Detection via Multi-Task Instruction Fine-Tuning. In *Findings of the Association for Computational Linguistics ACL 2024*. 10507–10521.
 - [30] Xu Duan, Jingzheng Wu, Shouling Ji, Zhiqing Rui, Tianyue Luo, Mutian Yang, and Yanjun Wu. 2019. VulSniper: Focus your attention to shoot fine-grained vulnerabilities. In *Proceedings of the 2019 International Joint Conference on Artificial Intelligence (IJCAI'19)*. 4665–4671.
 - [31] Natalie Dullerud, Karsten Roth, Kimia Hamidieh, Nicolas Papernot, and Marzyeh Ghassemi. [n.d.]. Is Fairness Only Metric Deep? Evaluating and Addressing Subgroup Gaps in Deep Metric Learning. In *International Conference on Learning Representations*.
 - [32] Jiahao Fan, Yi Li, Shaohua Wang, and Tien N. Nguyen. 2020. A C/C++ Code Vulnerability Dataset with Code Changes and CVE Summaries. In *Proceedings of the 17th International Conference on Mining Software Repositories (MSR'2020)*. 508–512.
 - [33] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155* (2020).
 - [34] Linton C. Freeman. 1978. Centrality in social networks conceptual clarification. *Social Networks* 1, 3 (1978), 215–239.
 - [35] Rikhiya Ghosh, Oladimeji Farri, Hans-Martin von Stockhausen, Martin Schmitt, and George Marica Vasile. 2024. CVE-LLM: Automatic vulnerability evaluation in medical device industry using large language models. *arXiv preprint arXiv:2407.14640* (2024).
 - [36] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified Cross-Modal Pre-training for Code Representation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 7212–7225.
 - [37] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, LIU Shujie, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, et al. [n.d.]. GraphCodeBERT: Pre-training Code Representations with Data Flow. In *International Conference on Learning Representations*.
 - [38] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence. <https://api.semanticscholar.org/CorpusID:267211867>
 - [39] Yutao Hu, Deqing Zou, Junru Peng, Yueming Wu, Junjie Shan, and Hai Jin. 2022. TreeCen: Building tree graph for scalable semantic code clone detection. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–12.
 - [40] Shin Hwei Tan Jie Wang Yuzhe Liu Hejun Fang Chunwei Xia Zheng Wang Huanting Wang, Zhanyong Tang. 2024. Combining Structured Static Code Information and Dynamic Symbolic Traces for Software Vulnerability Prediction. In *The IEEE/ACM 46th International Conference on Software Engineering (ICSE)*.
 - [41] Binyuan Hui, Jian Yang, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. 2024. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186* (2024).
 - [42] Jiyong Jang, Abeer Agrawal, and David Brumley. 2012. ReDeBug: Finding unpatched code clones in entire OS distributions. In *Proceedings of the 2012 IEEE Symposium on Security and Privacy (S&P'12)*. 48–62.
 - [43] Xuehai Jia, Junwei Du, Minying Fang, Hao Liu, Yuying Li, and Feng Jiang. 2025. Vulnerability detection with graph enhancement and global dependency representation learning. *Automated Software Engineering* 32, 1 (2025), 14.
 - [44] Lingxiao Jiang, Ghassan Mishnerghi, Zhendong Su, and Stephane Glondou. 2007. Deckard: Scalable and accurate tree-based detection of code clones. In *Proceedings of the 29th International Conference on Software Engineering (ICSE'07)*. 96–105.
 - [45] Haolin Jin, Linghan Huang, Haipeng Cai, Jun Yan, Bo Li, and Huaming Chen. 2024. From LLMs to LLM-based Agents for Software Engineering: A Survey of Current, Challenges and Future. *arXiv preprint arXiv:2408.02479* (2024).
 - [46] Toshihiro Kamiya, Shinji Kusumoto, and Katsuro Inoue. 2002. CCFinder: A multilingualistic token-based code clone detection system for large scale source code. *IEEE Transactions on Software Engineering* 28, 7 (2002), 654–670.
 - [47] Leo Katz. 1953. A new status index derived from sociometric analysis. *Psychometrika* 18, 1 (1953), 39–43.
 - [48] Seulbae Kim, Seunghoon Woo, Heejo Lee, and Hakjoo Oh. 2017. VUDDY: A scalable approach for vulnerable code clone discovery. In *Proceedings of the 2017 IEEE Symposium on Security and Privacy (S&P'17)*. 595–614.

- [49] Diederik P. Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).
- [50] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. 2012. Imagenet classification with deep convolutional neural networks. In *Proceedings of the 2012 Advances in Neural Information Processing Systems (NIPS'12)*. 1097–1105.
- [51] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. Gradient-based learning applied to document recognition. *IEEE* 86, 11 (1998), 2278–2324.
- [52] Bin Lei, Yuchen Li, and Qiuwu Chen. 2024. AutoCoder: Enhancing Code Large Language Model with\textsc{AIEV-Instruct}. *arXiv preprint arXiv:2405.14906* (2024).
- [53] Jingyue Li and Michael D. Ernst. 2012. CBCD: Cloned buggy code detector. In *Proceedings of the 34th International Conference on Software Engineering (ICSE'12)*. 310–320.
- [54] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. StarCoder: may the source be with you! *arXiv preprint arXiv:2305.06161* (2023).
- [55] Zhen Li, Deqing Zou, Shouhuai Xu, Zhaoxuan Chen, Yawei Zhu, and Hai Jin. 2021. Vuldelocator: a deep learning-based fine-grained vulnerability detector. *IEEE Transactions on Dependable and Secure Computing* (2021), 1–17.
- [56] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Hanchao Qi, and Jie Hu. 2016. VulPecker: An automated vulnerability detection system based on code similarity analysis. In *Proceedings of the 32nd Annual Conference on Computer Security Applications (ACSAC'16)*. 201–213.
- [57] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Yawei Zhu, and Zhaoxuan Chen. 2021. SySeVR: A framework for using deep learning to detect software vulnerabilities. *IEEE Transactions on Dependable and Secure Computing* (2021), 1–15.
- [58] Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, and Yuyi Zhong. 2018. VulDeePecker: A deep learning-based system for vulnerability detection. In *Proceedings of the 2018 Network and Distributed System Security Symposium (NDSS'18)*. 1–15.
- [59] Guanjun Lin, Jun Zhang, Wei Luo, Lei Pan, and Yang Xiang. 2017. POSTER: Vulnerability discovery with function representation learning from unlabeled projects. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS'17)*. 2539–2541.
- [60] Yu Liu, Lang Gao, Mingxin Yang, Yu Xie, Ping Chen, Xiaojin Zhang, and Wei Chen. 2024. VulDetectBench: Evaluating the Deep Capability of Vulnerability Detection with Large Language Models. *arXiv preprint arXiv:2406.07595* (2024).
- [61] Zhuang Liu and et al. 2022. A ConvNet for the 2020s. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 11976–11986.
- [62] Zhongxin Liu, Zhijie Tang, Junwei Zhang, Xin Xia, and Xiaohu Yang. 2024. Pre-training by Predicting Program Dependencies for Vulnerability Analysis Tasks. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (Lisbon, Portugal) (ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 151, 13 pages. <https://doi.org/10.1145/3597503.3639142>
- [63] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, et al. 2024. Starcoder 2 and the stack v2: The next generation. *arXiv preprint arXiv:2402.19173* (2024).
- [64] Guilong Lu, Xiaolin Ju, Xiang Chen, Wenlong Pei, and Zhilong Cai. 2024. GRACE: Empowering LLM-based software vulnerability detection with graph structure and in-context learning. *Journal of Systems and Software* 212 (2024), 112031.
- [65] Andrew A Mahyari. 2024. Harnessing the Power of LLMs in Source Code Vulnerability Detection. *arXiv preprint arXiv:2408.03489* (2024).
- [66] Massimo Marchiori and Vito Latora. 2000. Harmony in the small-world. *Physica A: Statistical Mechanics and its Applications* 285, 3–4 (2000), 539–546.
- [67] Daye Nam, Andrew Macvean, Vincent Hellendoorn, Bogdan Vasilescu, and Brad Myers. 2024. Using an llm to help with code understanding. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.
- [68] Stephan Neuhaus, Thomas Zimmermann, Christian Holler, and Andreas Zeller. 2007. Predicting vulnerable software components. In *Proceedings of the 14th ACM Conference on Computer and Communications Security (CCS'07)*. 529–540.
- [69] Van-Anh Nguyen, Dai Quoc Nguyen, Van Nguyen, Trung Le, Quan Hung Tran, and Dinh Phung. 2022. ReGVD: Revisiting Graph Neural Networks for Vulnerability Detection. In *Proceedings of the 44th International Conference on Software Engineering Companion (ICSE '22 Companion)*.
- [70] Matteo Pagliardini, Prakhar Gupta, and Martin Jaggi. 2017. Unsupervised learning of sentence embeddings using compositional n-gram features. *arXiv preprint arXiv:1703.02507* (2017).
- [71] Nam H. Pham, Tung Thanh Nguyen, Hoan Anh Nguyen, and Tien N. Nguyen. 2010. Detection of recurring software vulnerabilities. In *Proceedings of the 2010 International Conference on Automated Software Engineering (ASE'10)*. 447–456.
- [72] Nikhil Pinnaparaju, Reshith Adithyan, Duy Phung, Jonathan Tow, James Baicoianu, Ashish Datta, Maksym Zhuravinskiy, Dakota Mahan, Marco Bellagente, Carlos Riquelme, et al. 2024. Stable code technical report. *arXiv preprint arXiv:2404.01226* (2024).
- [73] Fangcheng Qiu, Zhongxin Liu, Xing Hu, Xin Xia, Gang Chen, and Xinyu Wang. 2024. Vulnerability Detection via Multiple-Graph-Based Code Representation. *IEEE Trans. Softw. Eng.* 50, 8 (Aug. 2024), 2178–2199. <https://doi.org/10.1109/TSE.2024.3427815>
- [74] Radim Rehurek, Petr Sojka, et al. 2011. Gensim—statistical semantics in python. *Retrieved from genism.org* (2011).
- [75] Yannick Rochat. 2009. Closeness centrality extended to unconnected graphs: The harmonic centrality index. In *Asna*.

- [76] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).
- [77] Rebecca Russell, Louis Kim, Lei Hamilton, Tomo Lazovich, Jacob Harer, Onur Ozdemir, Paul Ellingwood, and Marc McConley. 2018. Automated vulnerability detection in source code using deep representation learning. In *Proceedings of the 2018 IEEE International Conference on Machine Learning and Applications (ICMLA'18)*. 757–762.
- [78] Hitesh Sajjani, Vaibhav Saini, Jeffrey Svajlenko, Chanchal K. Roy, and Cristina V. Lopes. 2016. SourcererCC: Scaling code clone detection to big-code. In *Proceedings of the 38th International Conference on Software Engineering (ICSE'16)*. 1157–1168.
- [79] Umesh Shankar, Kunal Talwar, Jeffrey S. Foster, and David A. Wagner. 2001. Detecting format string vulnerabilities with type qualifiers. In *Proceedings of the 2001 USENIX Security Symposium (USENIX Security'01)*. 201–220.
- [80] Lwin Khin Shar, Lionel C. Briand, and Hee Beng Kuan Tan. 2014. Web application vulnerability prediction using hybrid program analysis and machine learning. *IEEE Transactions on Dependable and Secure Computing* 12, 6 (2014), 688–707.
- [81] Aleksei Shestov, Rodion Levichev, Ravil Mussabayev, Evgeny Maslov, Pavel Zadorozhny, Anton Cheshkov, Rustam Mussabayev, Alymzhan Toleu, Gulmira Tolegen, and Alexander Krassovitskiy. 2025. Finetuning Large Language Models for Vulnerability Detection. *IEEE Access* 13 (2025), 38889–38900. <https://doi.org/10.1109/ACCESS.2025.3546700>
- [82] Sadegh Soleimani, Mahsa Bahrami, and Mansour Vali. 2025. Survival prediction from imbalanced colorectal cancer dataset using hybrid sampling methods and tree-based classifiers. *Scientific Reports* 15, 1 (2025), 1–13.
- [83] Mingxing Tan and Quoc Le. 2019. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. In *International Conference on Machine Learning*. PMLR, 6105–6114.
- [84] Xiaoyu Tan, Shaojie Shi, Xihe Qiu, Chao Qu, Zhenting Qi, Yinghui Xu, and Yuan Qi. 2023. Self-Criticism: Aligning Large Language Models with their Understanding of Helpfulness, Honesty, and Harmlessness. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: Industry Track*, Mingxuan Wang and Imed Zitouni (Eds.). Association for Computational Linguistics, Singapore, 650–662. <https://doi.org/10.18653/v1/2023.emnlp-industry.62>
- [85] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).
- [86] Shubham Ugare, Tarun Suresh, Hangoo Kang, Sasa Misailovic, and Gagandeep Singh. 2024. Improving llm code generation with grammar augmentation. *arXiv preprint arXiv:2403.01632* (2024).
- [87] MC Wang and WF Pan. 2012. A comparative study of network centrality metrics in identifying key classes in software. *Journal of Computational Information Systems* 8, 24 (2012), 10205–10212.
- [88] Yue Wang, Hung Le, Akhilesh Deepak Gotmare, Nghi D.Q. Bui, Junnan Li, and Steven C. H. Hoi. 2023. CodeT5+: Open Code Large Language Models for Code Understanding and Generation. *arXiv preprint* (2023).
- [89] Xin-Cheng Wen, Yupan Chen, Cuiyun Gao, Hongyu Zhang, Jie M Zhang, and Qing Liao. 2023. Vulnerability Detection with Graph Simplification and Enhanced Graph Representation Learning. In *2023 IEEE/ACM 44th International Conference on Software Engineering (ICSE)*.
- [90] Yueming Wu, Xiaodi Li, Deqing Zou, Wei Yang, Xin Zhang, and Hai Jin. 2019. MalScan: Fast market-wide mobile malware scanning by social-network centrality analysis. In *Proceedings of the 34th International Conference on Automated Software Engineering (ASE'19)*. 139–150.
- [91] Yueming Wu, Deqing Zou, Shihan Dou, Wei Yang, Duo Xu, and Hai Jin. 2022. VulCNN: An Image-inspired Scalable Vulnerability Detection System. In *2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE)*. 2365–2376. <https://doi.org/10.1145/3510003.3510229>
- [92] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S. Yu. 2021. A Comprehensive Survey on Graph Neural Networks. *IEEE Transactions on Neural Networks and Learning Systems* 32, 1 (2021), 4–24. <https://doi.org/10.1109/TNNLS.2020.2978386>
- [93] Fabian Yamaguchi, Nico Golde, Daniel Arp, and Konrad Rieck. 2014. Modeling and discovering vulnerabilities with code property graphs. In *Proceedings of the 2014 IEEE Symposium on Security and Privacy (S&P'14)*. 590–604.
- [94] Fabian Yamaguchi, Markus Lottmann, and Konrad Rieck. 2012. Generalized vulnerability extrapolation using abstract syntax trees. In *Proceedings of the 28th Annual Computer Security Applications Conference (ACSAC'12)*. 359–368.
- [95] Fabian Yamaguchi, Alwin Maier, Hugo Gascon, and Konrad Rieck. 2015. Automatic inference of search patterns for taint-style vulnerabilities. In *Proceedings of the 2015 IEEE Symposium on Security and Privacy (S&P'15)*. 797–812.
- [96] Aidan ZH Yang, Haoye Tian, He Ye, Ruben Martins, and Claire Le Goues. 2024. Security Vulnerability Detection with Multitask Self-Instructed Fine-Tuning of Large Language Models. *arXiv preprint arXiv:2406.05892* (2024).
- [97] Yanjing Yang, Xin Zhou, Runfeng Mao, Jinwei Xu, Lanxin Yang, Yu Zhangm, Haifeng Shen, and He Zhang. 2024. DLAP: A Deep Learning Augmented Large Language Model Prompting Framework for Software Vulnerability Detection. *arXiv preprint arXiv:2405.01202* (2024).
- [98] Junwei Zhang, Zhongxin Liu, Xing Hu, Xin Xia, and Shanping Li. 2023. Vulnerability detection by learning from syntax-based execution paths of code. *IEEE Transactions on Software Engineering* 49, 8 (2023), 4196–4212.

- [99] Rui Zheng, Shihan Dou, Songyang Gao, Yuan Hua, Wei Shen, Binghai Wang, Yan Liu, Senjie Jin, Qin Liu, Yuhao Zhou, et al. 2023. Secrets of rlhf in large language models part i: Ppo. *arXiv preprint arXiv:2307.04964* (2023).
- [100] Xin Zhou, Sicong Cao, Xiaobing Sun, and David Lo. 2024. Large language model for vulnerability detection and repair: Literature review and the road ahead. *ACM Transactions on Software Engineering and Methodology* (2024).
- [101] Xin Zhou, Ting Zhang, and David Lo. 2024. Large language model for vulnerability detection: Emerging results and future directions. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*. 47–51.
- [102] Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, and Yang Liu. 2019. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In *Proceedings of the 2019 Advances in Neural Information Processing Systems (NIPS'19)*. 10197–10207.
- [103] Deqing Zou, Sujuan Wang, Shouhuai Xu, Zhen Li, and Hai Jin. 2019. μ VulDeePecker: A deep learning-based system for multiclass vulnerability detection. *IEEE Transactions on Dependable and Secure Computing* 18, 5 (2019), 1–13.